

L8: Blocks and Functions in CSE Machine

CS1101S: Programming Methodology

Martin Henz

October 11, 2023

Outline

- Recap: CSE Machine
- Blocks
- Functions
- Stateful Data and Identity

Announcements

- New **game chapters** are out in Source Academy
 - Back to a weekly schedule
- Winners of **Game of Tones** contest
 - Presented by Grandmaster Boyd

Outline

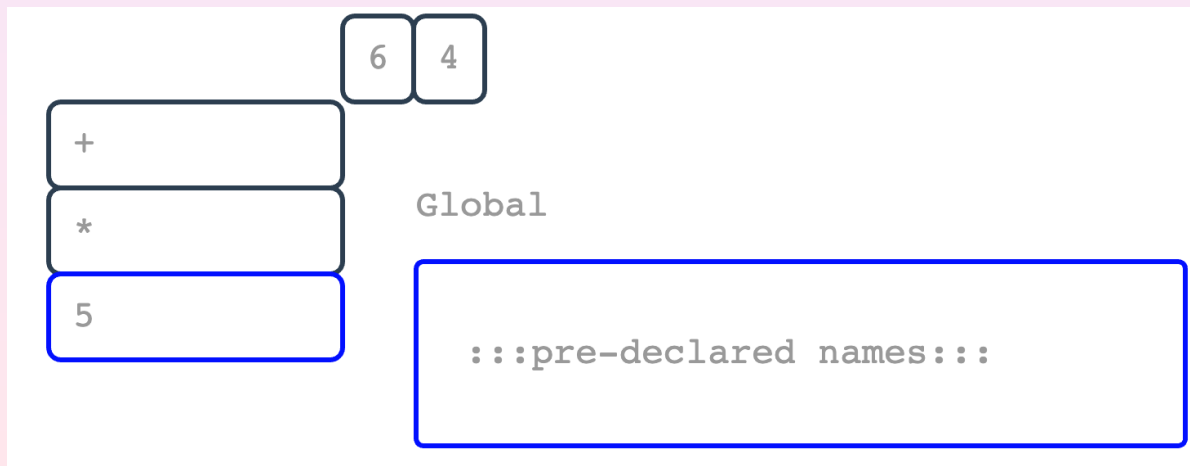
- Recap: CSE Machine
- Blocks
- Functions
- Stateful Data and Identity

Recap: expression evaluation in CSE machine

Control tracks deferred operations,
stash stores intermediate results

[Show in
Playground](#)

2 * 3 + 4 * 5;

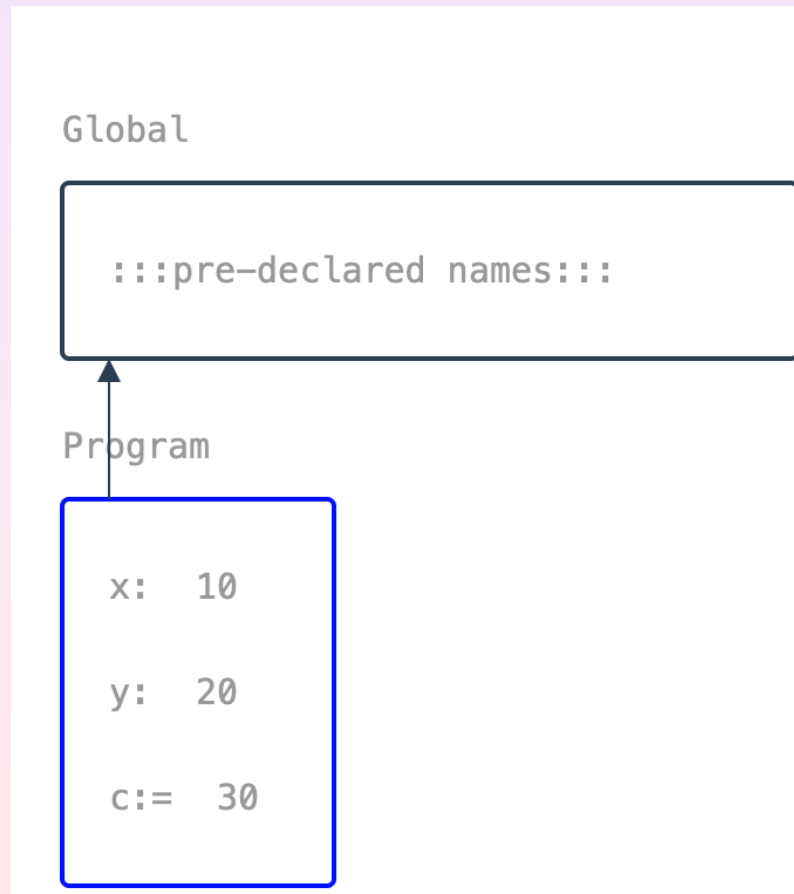


Recap: variable declaration in CSE machine

Environment provides locations for storing values for names

```
let x = 10;  
let y = 20;  
const c = 30;  
x = y + c;
```

[Show in
Playground](#)

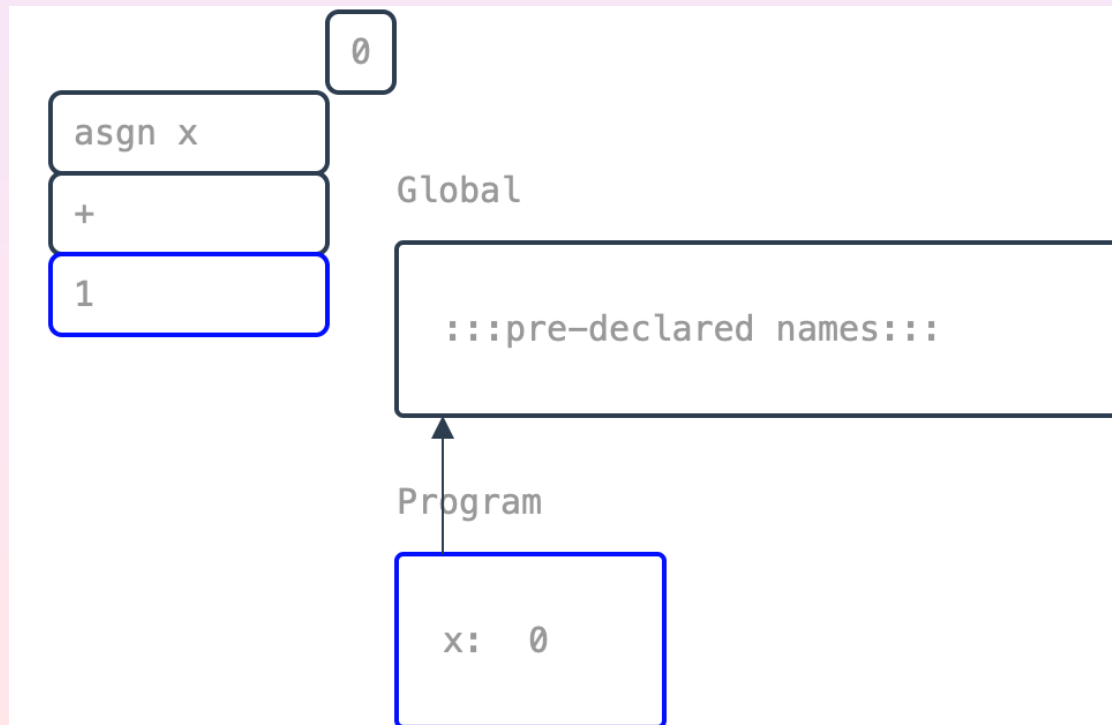


Recap: assignment in CSE machine

Control keep track of expressions and assignment,
stash stores intermediate values,
environment is where the change happens

[Show in
Playground](#)

```
let x = 0;  
x = x + 1;
```



Recap: environments in CSE machine

[Show in
Playground](#)

- An ***environment*** determines the context in which an expression should be evaluated
- Every **expression** is evaluated with respect to the **current** environment (indicated in blue)
- **Global environment** provides predeclared names
- **Program environment** provides locations for storing values for names declared in the program

Outline

- Recap: CSE Machine
- **Blocks**
- Functions
- Stateful Data and Identity

How will blocks work in CSE machine?

Example

[Show in
Playground](#)

```
const x = 1;  
{  
  const x = 6;  
  display(x);  
}  
x + 1;
```

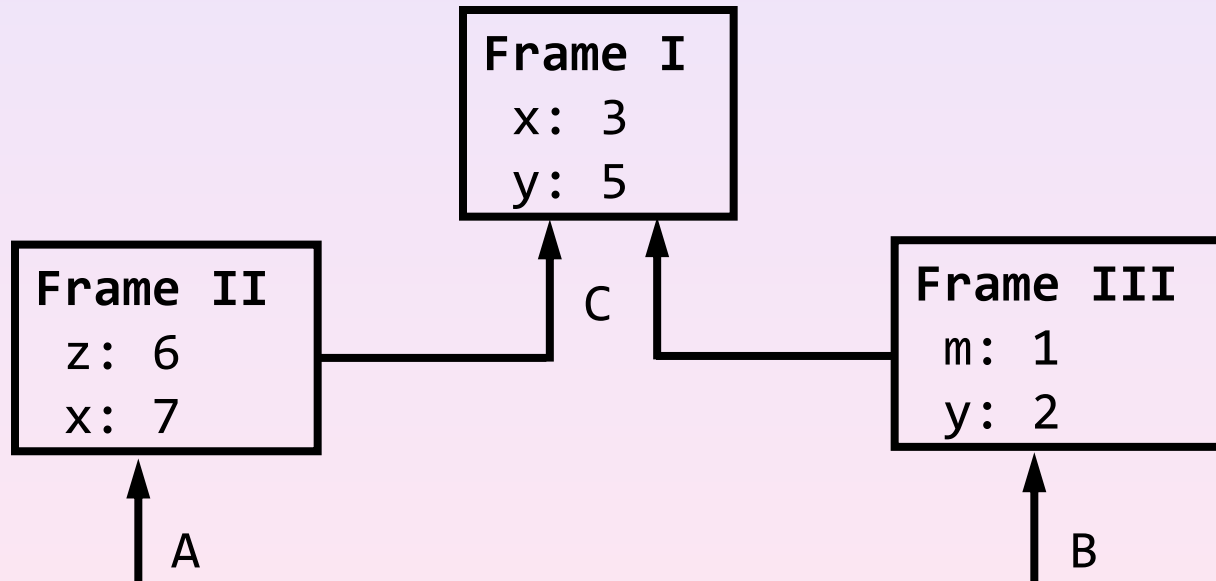
**In CSE machine: How to
keep track of different
x'es?**

**The substitution model
replaces x by 10 in `x + 1`;
but not in `display(x)`;**

Environments

- An **environment** is a **sequence of frames**
- Each frame contains **bindings** of values to names
- A frame points to its **enclosing environment**, the next one in the sequence
 - Unless the frame is the **global frame**
- **Extending** an environment means **adding** a new frame “inside” the old one

Example environments



- A, B, C are environments
- Value of **x** in environment **A** is **7**
- Value of **x** in environment **B** is **3**
- **x** in Frame II **shadows** **x** in Frame I

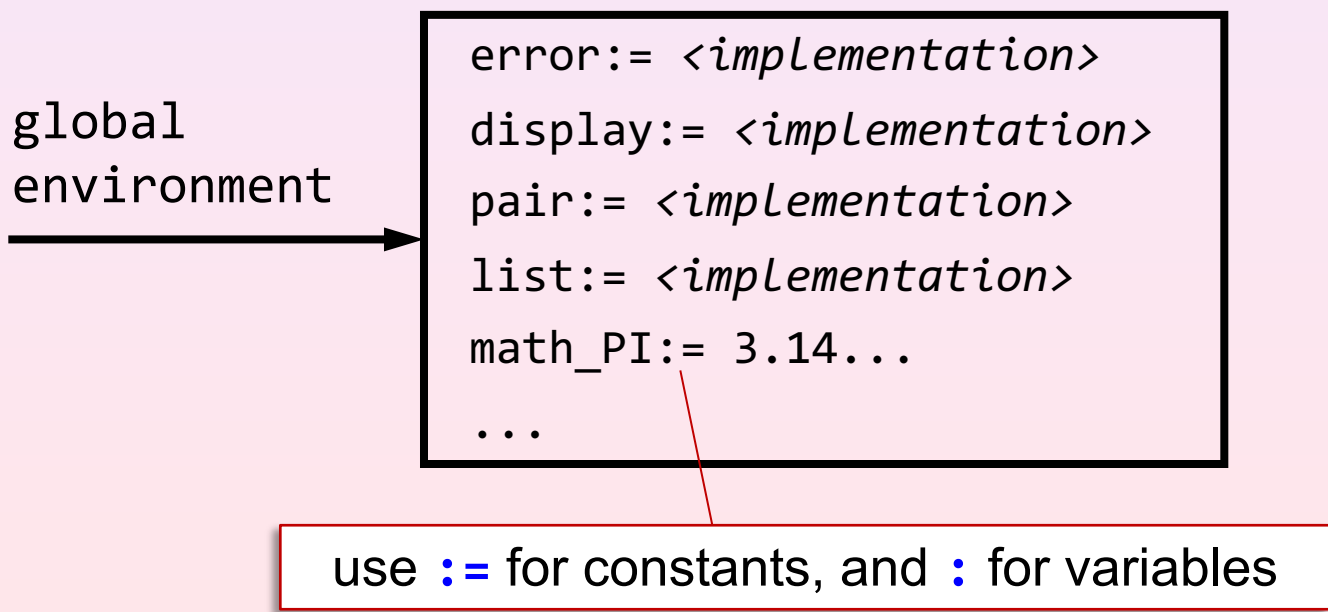
Accessing environment

- To evaluate a **name**, look up its value in the **current frame**
- If **not found in current frame**, then look in **enclosing environment**, and so on
- If **still not found?** **Well, that never happens!**

Global environment

- The **global environment** consists of a single **frame** with bindings of **primitive** and **pre-declared** functions and constants
 - Functions' names are bound to their implementations

global
environment



```
error:= <implementation>
display:= <implementation>
pair:= <implementation>
list:= <implementation>
math_PI:= 3.14...
...
```

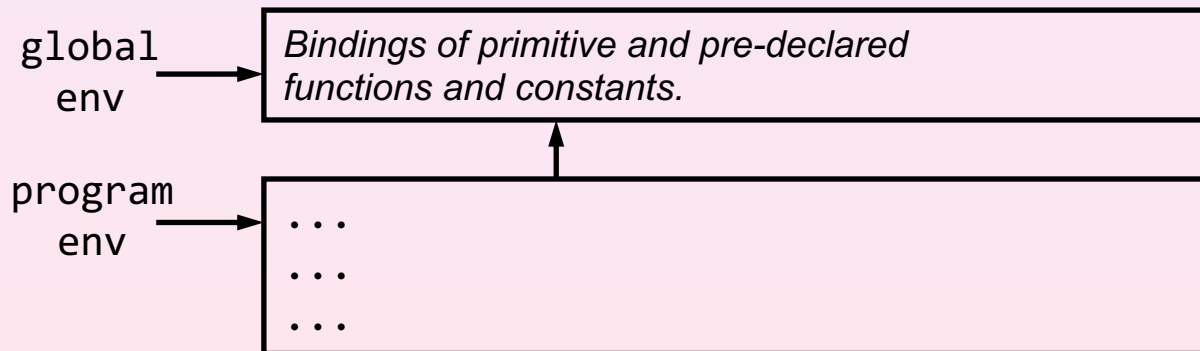
use **:=** for constants, and **:** for variables

Evaluating blocks

- Evaluating a **block** { *statements* } extends the current environment by adding a **new frame**
 - New frame contains bindings of the **constants** and **variables** declared in the block
 - Initially, these **constants** and **variables** don't have associated values (**unassigned**)
 - When the **declarations** are evaluated, the **names** are **assigned** their values
- **No frame is created** if the block has **no constant or variable declaration**
 - **There are no empty frames**

Evaluating blocks

- **User program** is considered in an implicit **program block**
 - Program evaluation directly extends the **global environment**
 - Resulting environment is called the **program environment**



Evaluating blocks: example

```
let x = 3;  
  
{  
  const x = 5;  
  let y = 8;  
}
```

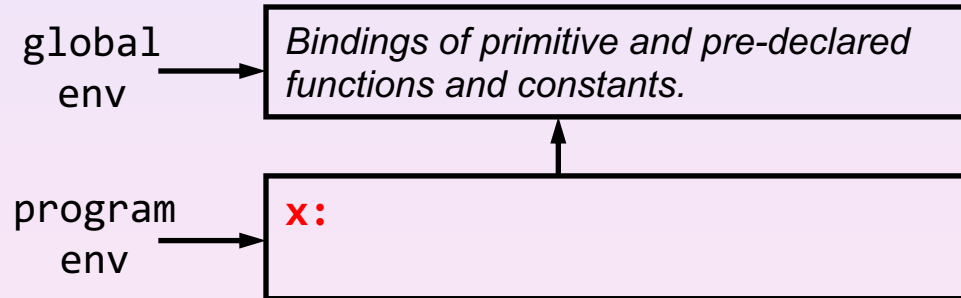
global
env

→ *Bindings of primitive and pre-declared
functions and constants.*

[Show in
Playground](#)

Evaluating blocks: example

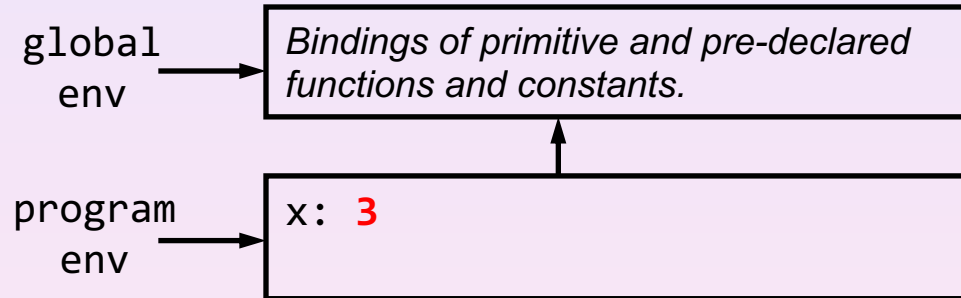
```
let x = 3;  
  
{  
  const x = 5;  
  let y = 8;  
}
```



[Show in
Playground](#)

Example for evaluating blocks

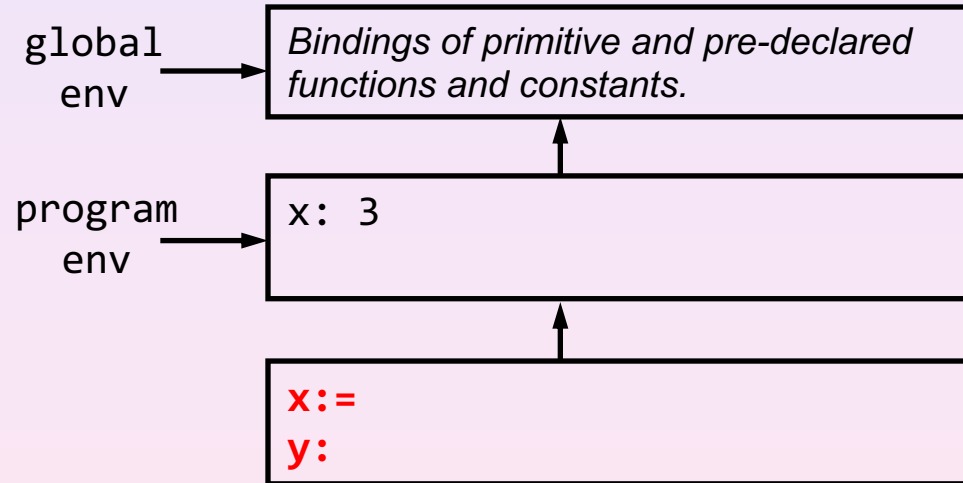
```
let x = 3;  
  
{  
  const x = 5;  
  let y = 8;  
}
```



[Show in
Playground](#)

Example for evaluating blocks

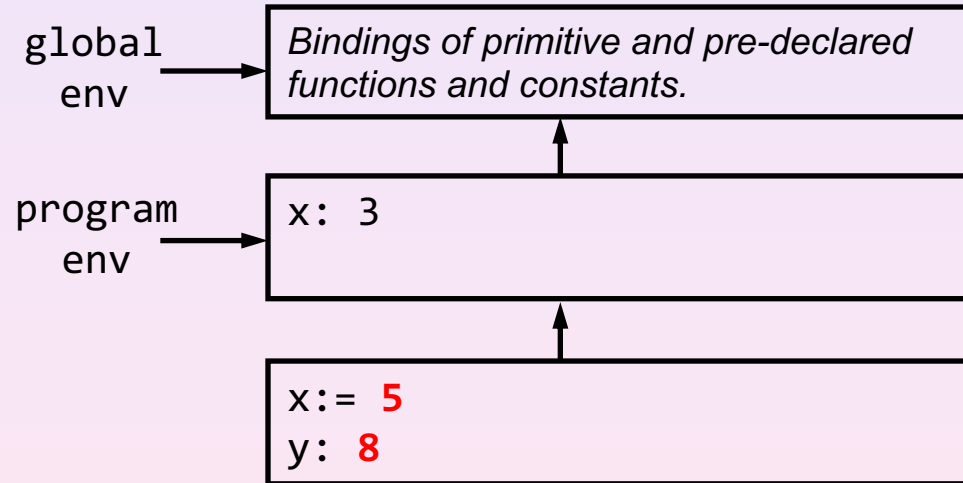
```
let x = 3;  
  
{  
  const x = 5;  
  let y = 8;  
}
```



[Show in Playground](#)

Example for evaluating blocks

```
let x = 3;  
  
{  
  const x = 5;  
  let y = 8;  
}
```



[Show in Playground](#)

Declarations, names, assignments: revisited

- **Constant / variable** declaration statements:

const name = value;

let name = value;

- Makes binding of *name* to *value* in the **current frame**

- **Name occurrence:**

name

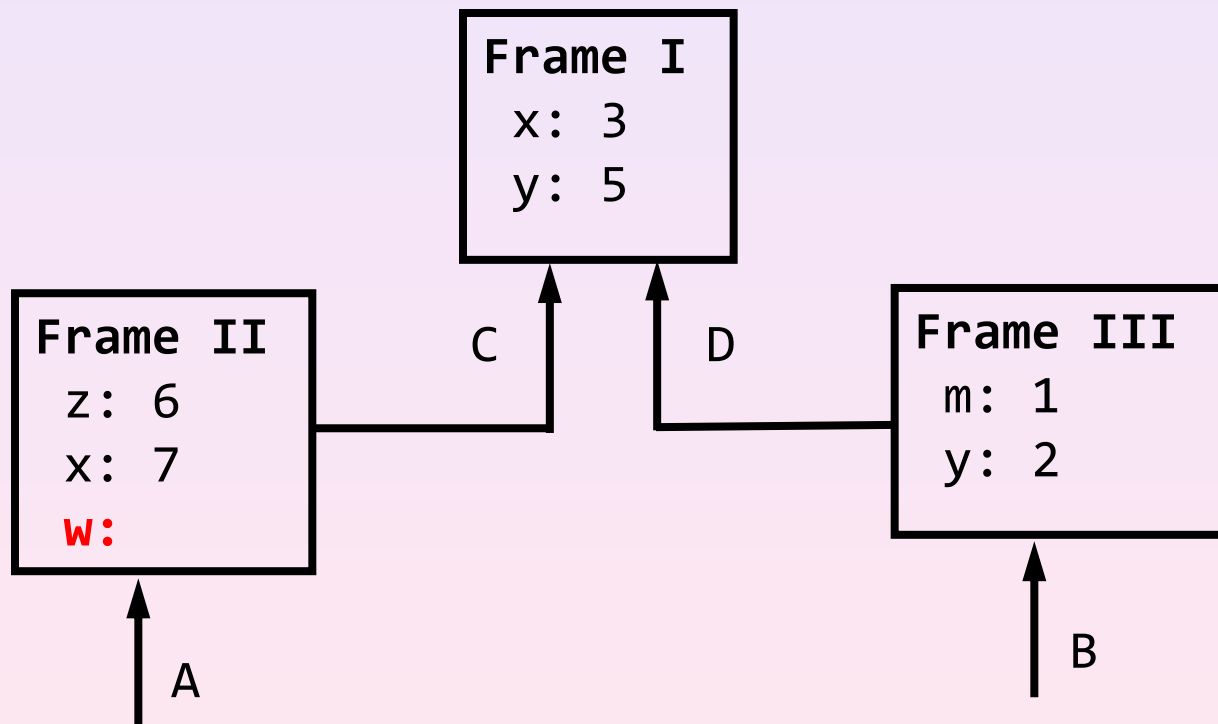
- Searches for *name* beginning from **current frame**, returns value
 - If *name* is **unassigned**, signal an error

- **Assignment** statement:

name = new_value;

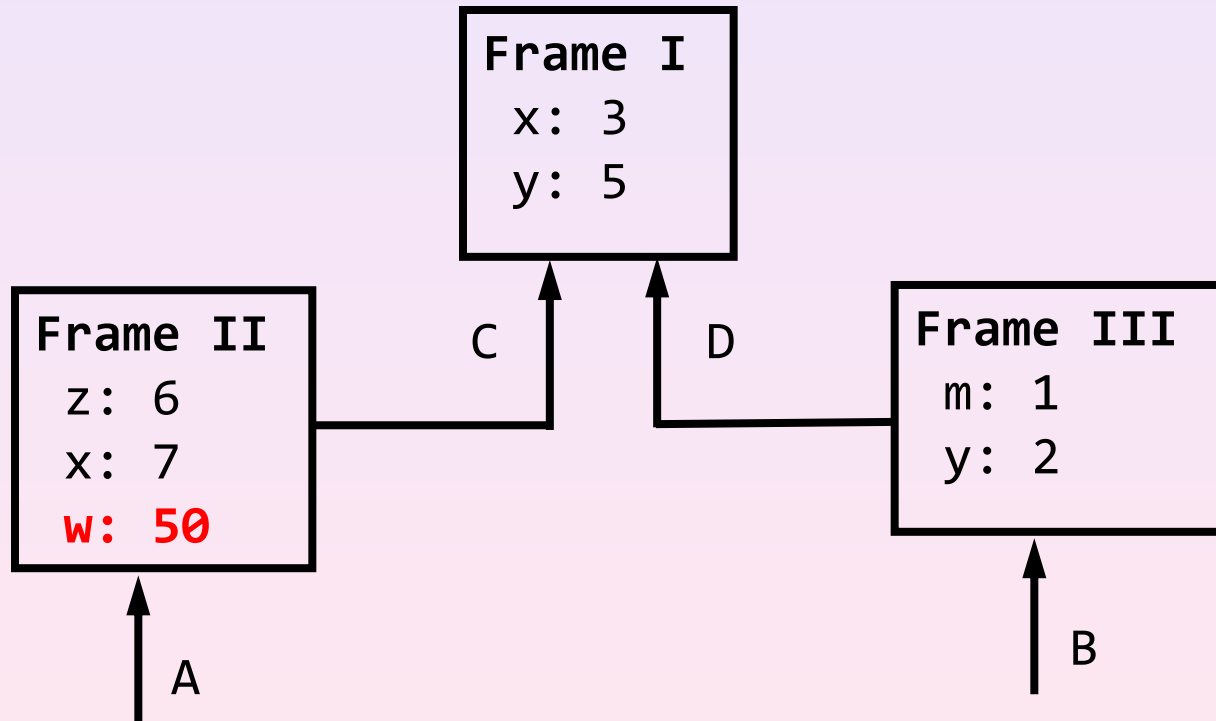
- Searches for *name* beginning from **current frame**, then changes its binding to *new_value*
 - If *name* is **unassigned**, signal an error

Example environments



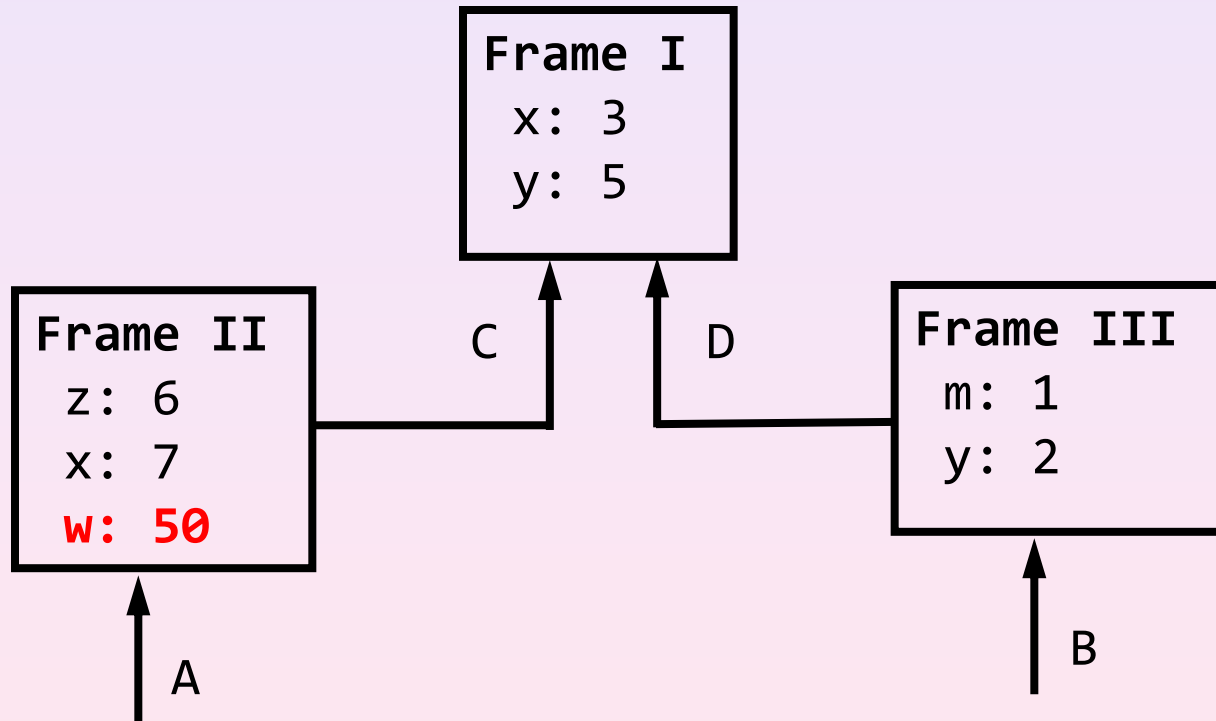
- Evaluate in **A**: `let w = 50;`

Example environments



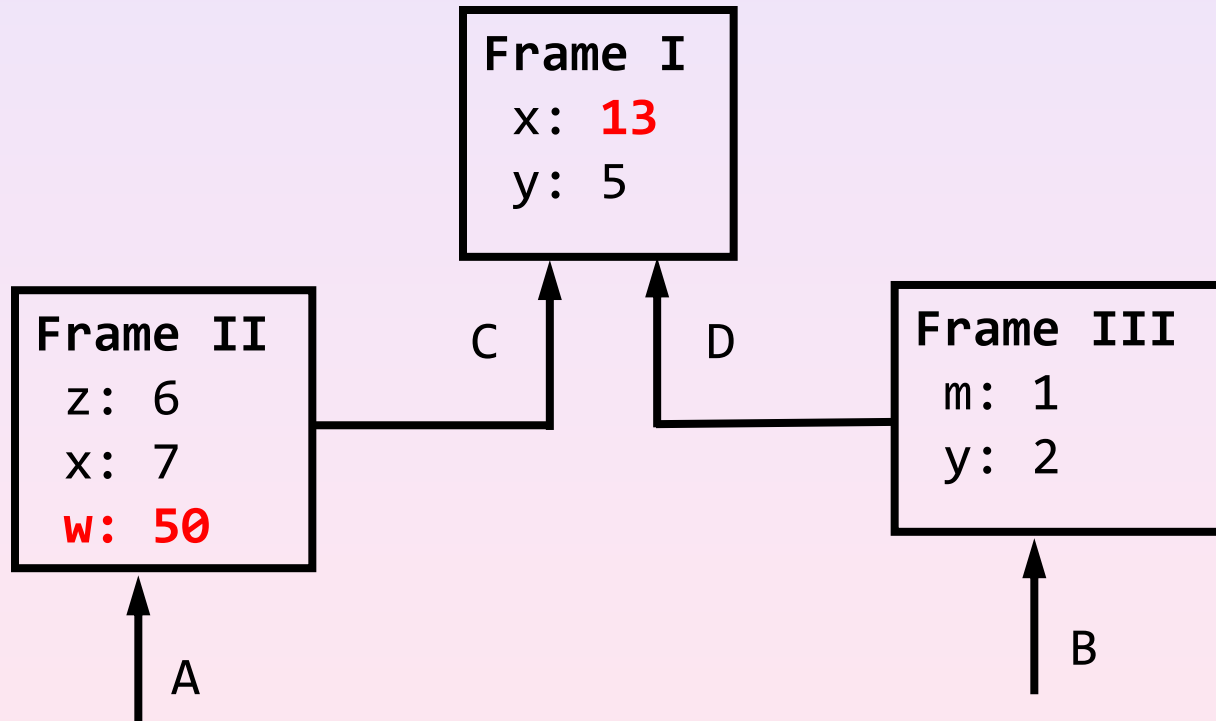
- Evaluate in **A**: `let w = 50;`
 - `w` in Frame II is assigned with the value **50**

Example environments



- Evaluate in **B**: $x = x + 10$;

Example environments



- Evaluate in **B**: $x = x + 10$;
 - The value of x in Frame I is updated from 3 to 13

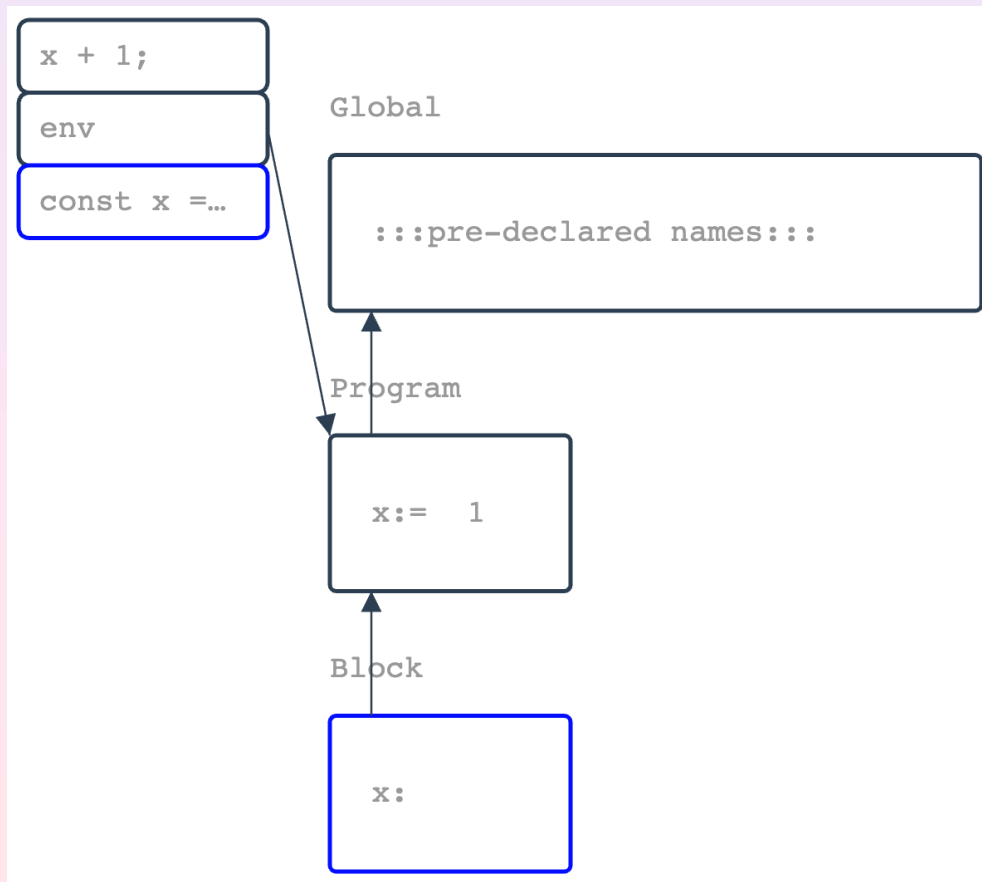
A small difficulty

```
const x = 1;  
{  
  const x = 6; ...  
}  
x + 1;
```

[Show in
Playground](#)

Problem: How do we go
back to the previous
environment in the CSE
machine?

Solution: env instructions

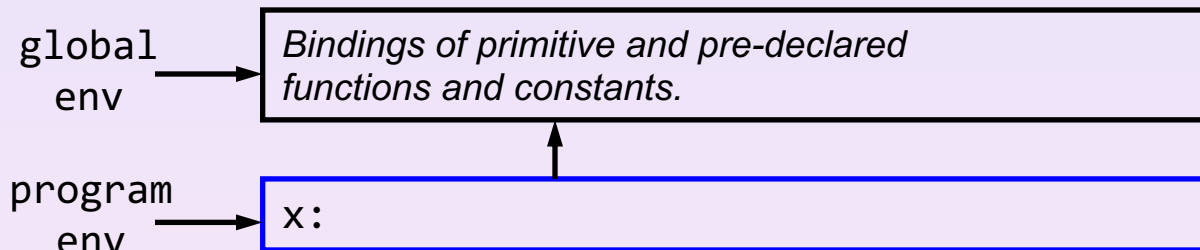


Block example

[Show in
Playground](#)

```
let x = 3;

if (x <= 1) {
  let y = x;
  x = y + 1;
} else {
  let z = x;
  x = z * z;
}
{
  x = x + 1;
}
{
  let x = 5;
  let y = 8;
  {
    let x = 20 + y;
  }
  x = x + 1;
}
x;
```

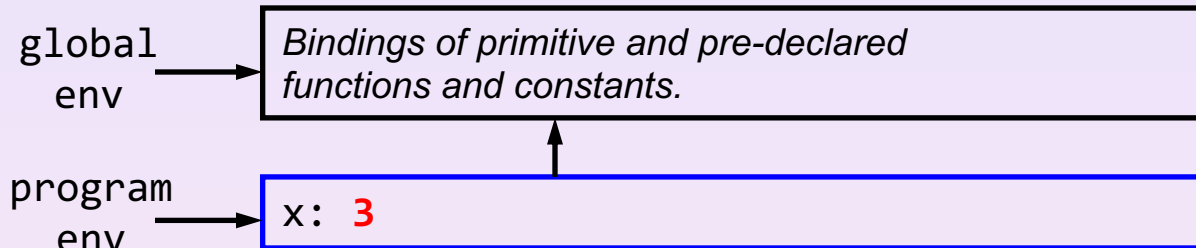


```
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;
```

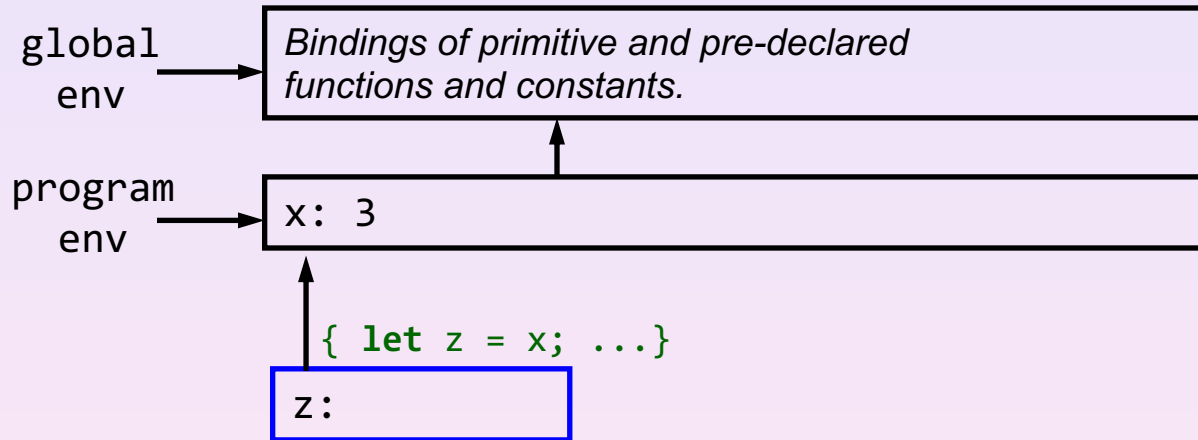


```
let x = 3;

if (x <= 1) {
  let y = x;
  x = y + 1;
} else {
  let z = x;
  x = z * z;
}

{
  x = x + 1;
}

{
  let x = 5;
  let y = 8;
  {
    let x = 20 + y;
  }
  x = x + 1;
}
x;
```

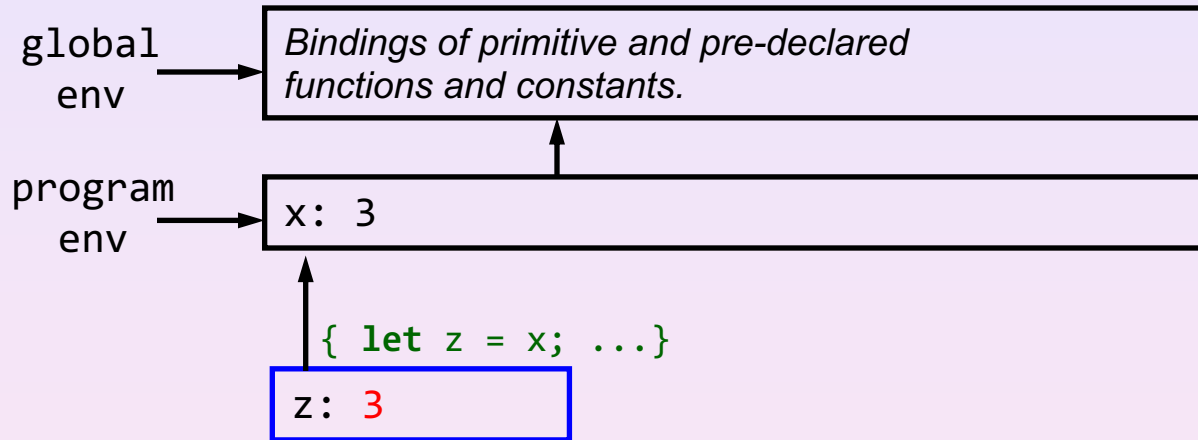


```
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;
```

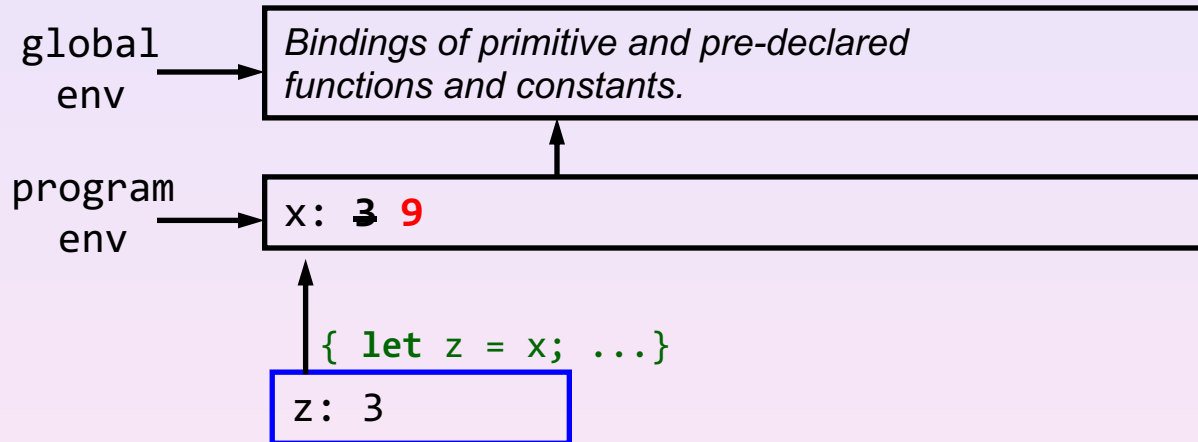


```
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;
```

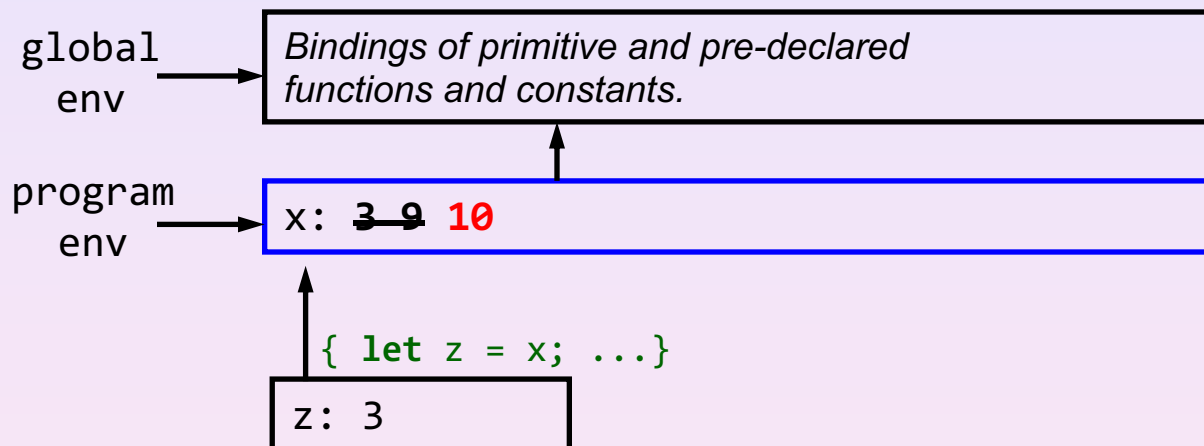



```
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;
```

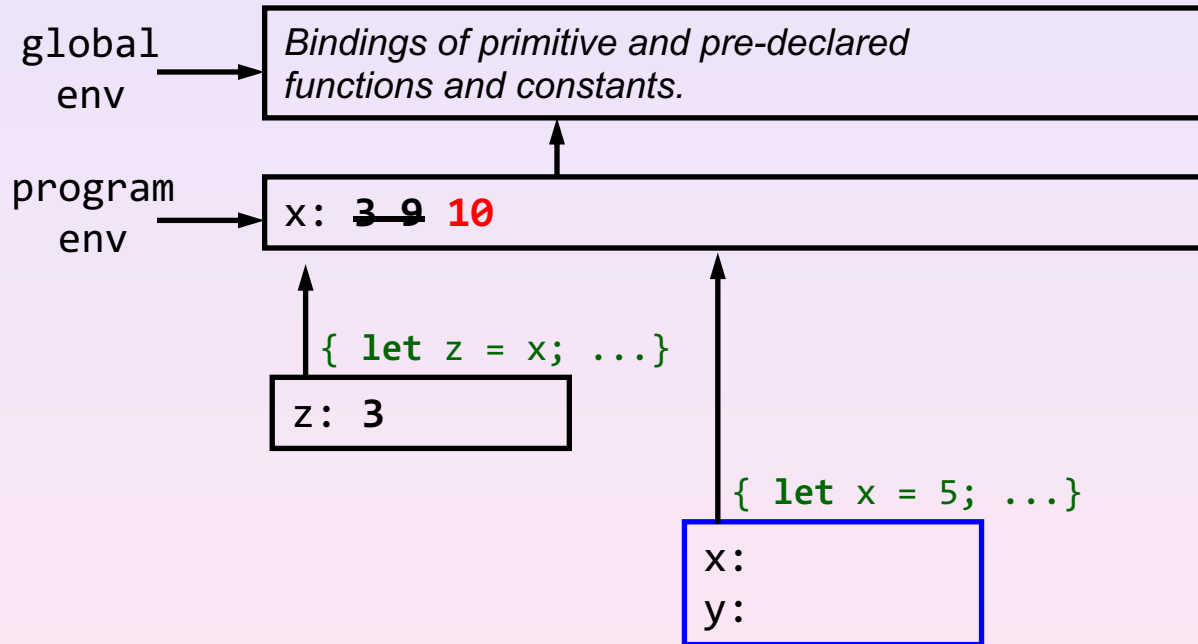


```
let x = 3;

if (x <= 1) {
  let y = x;
  x = y + 1;
} else {
  let z = x;
  x = z * z;
}

{
  x = x + 1;
}

{
  let x = 5;
  let y = 8;
  {
    let x = 20 + y;
  }
  x = x + 1;
}
x;
```



```

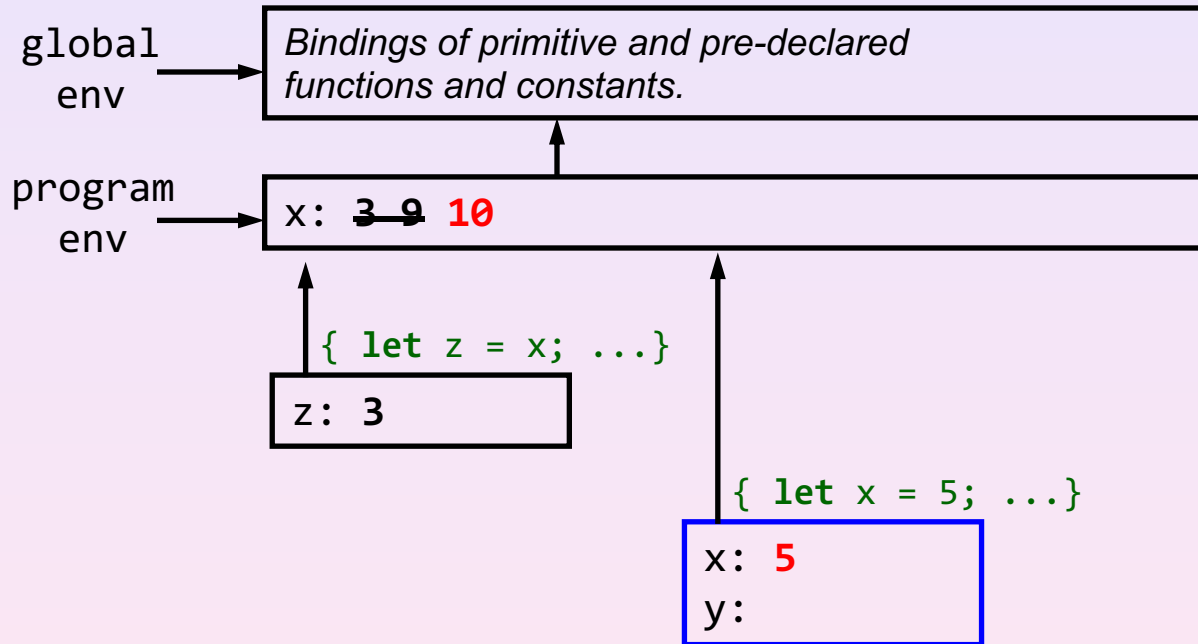
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;

```



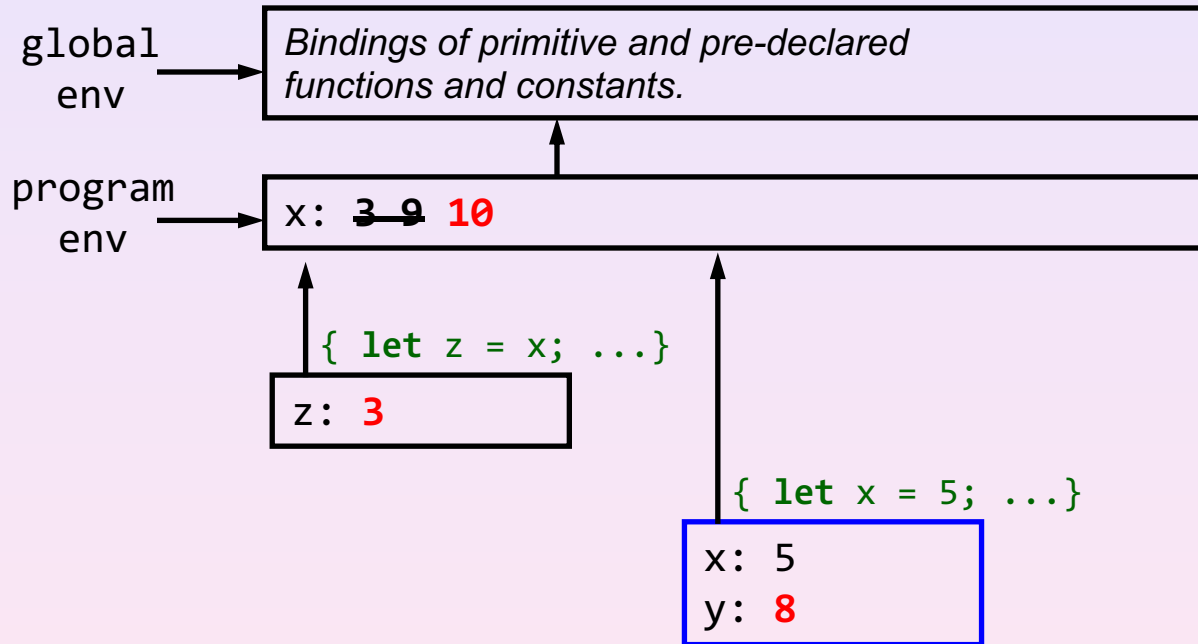
```

let x = 3;

if (x <= 1) {
  let y = x;
  x = y + 1;
} else {
  let z = x;
  x = z * z;
}

x = x + 1;

let x = 5;
let y = 8;
{
  let x = 20 + y;
}
x = x + 1;
x;
  
```



```

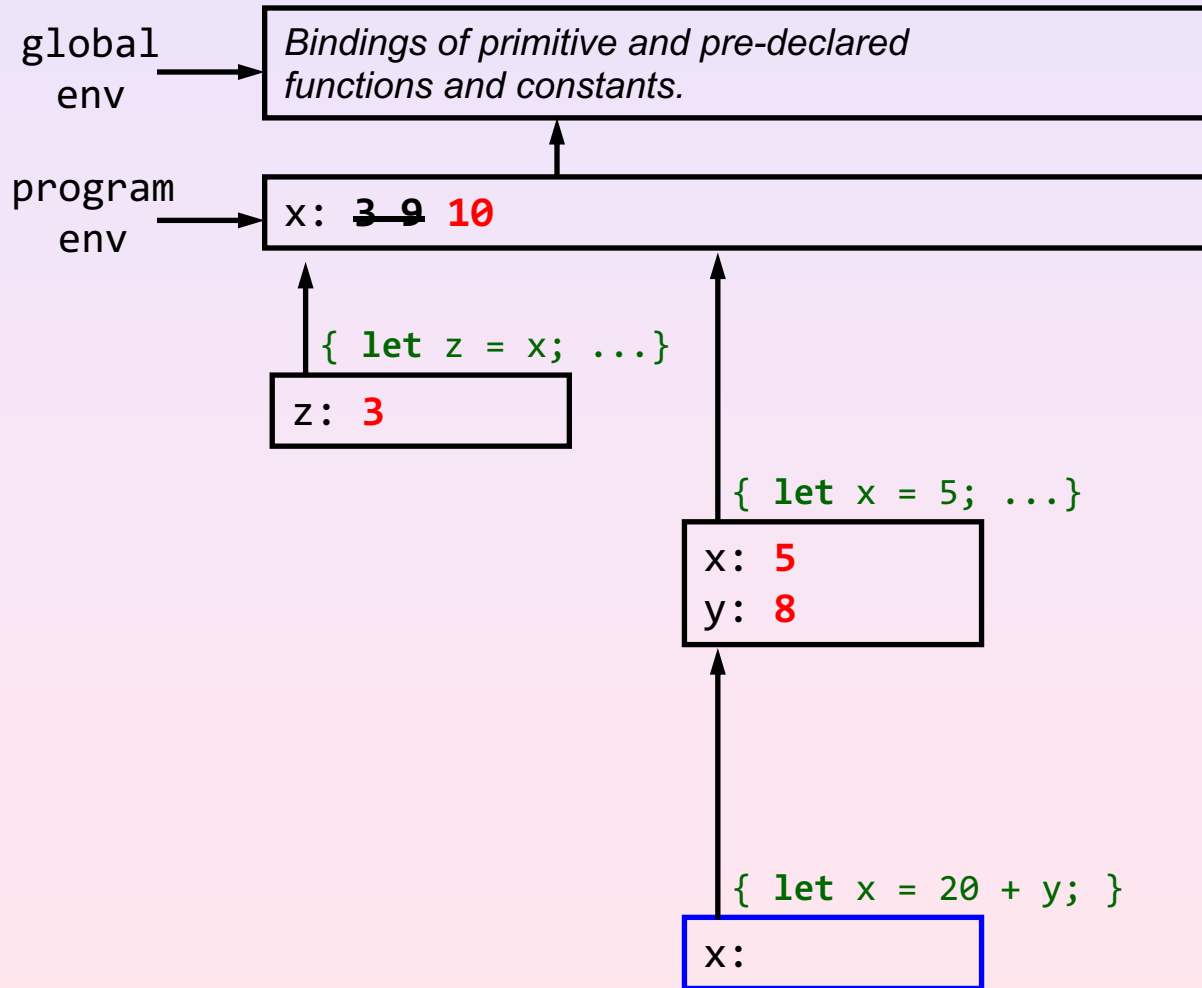
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;

```



```

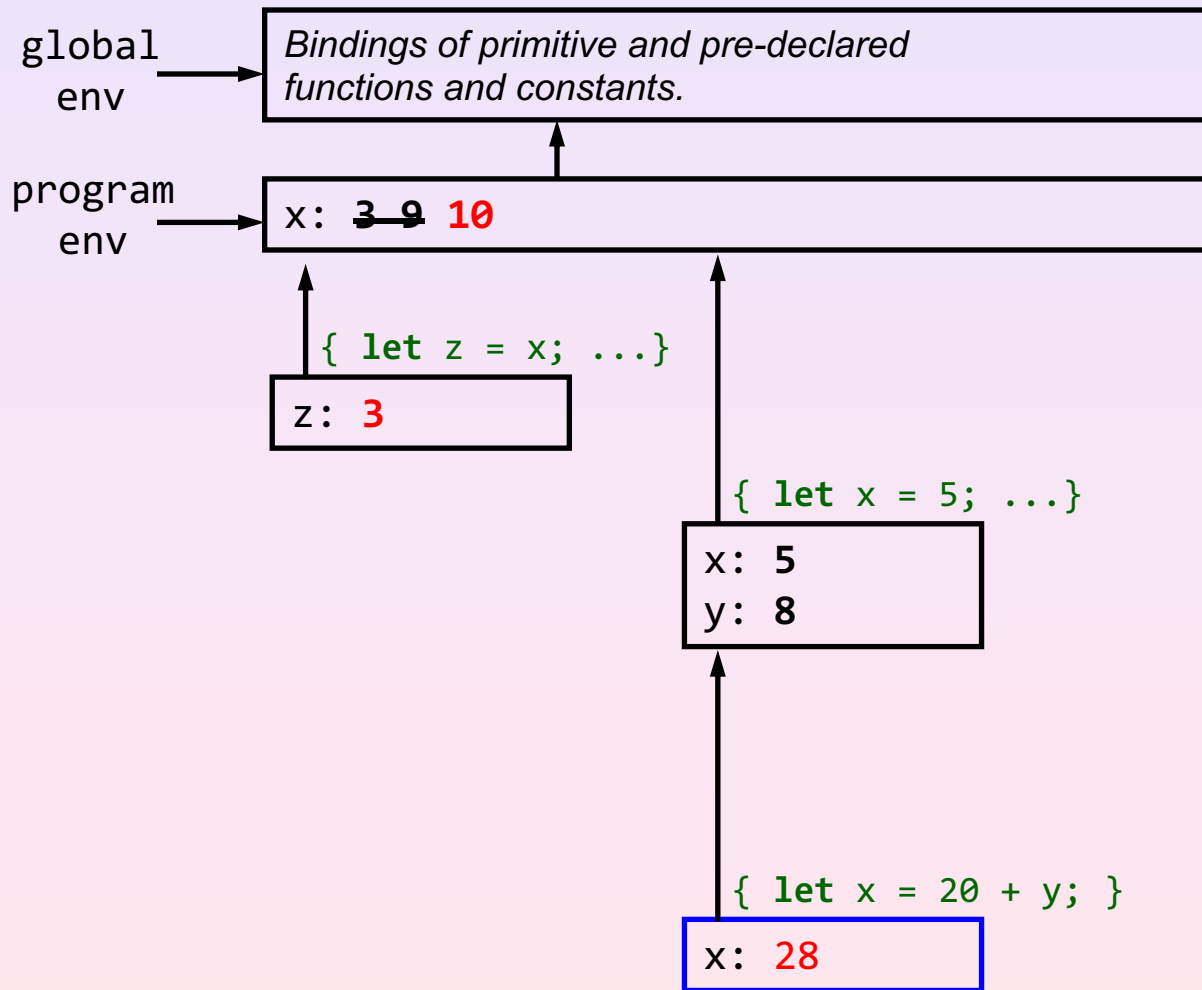
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;

```



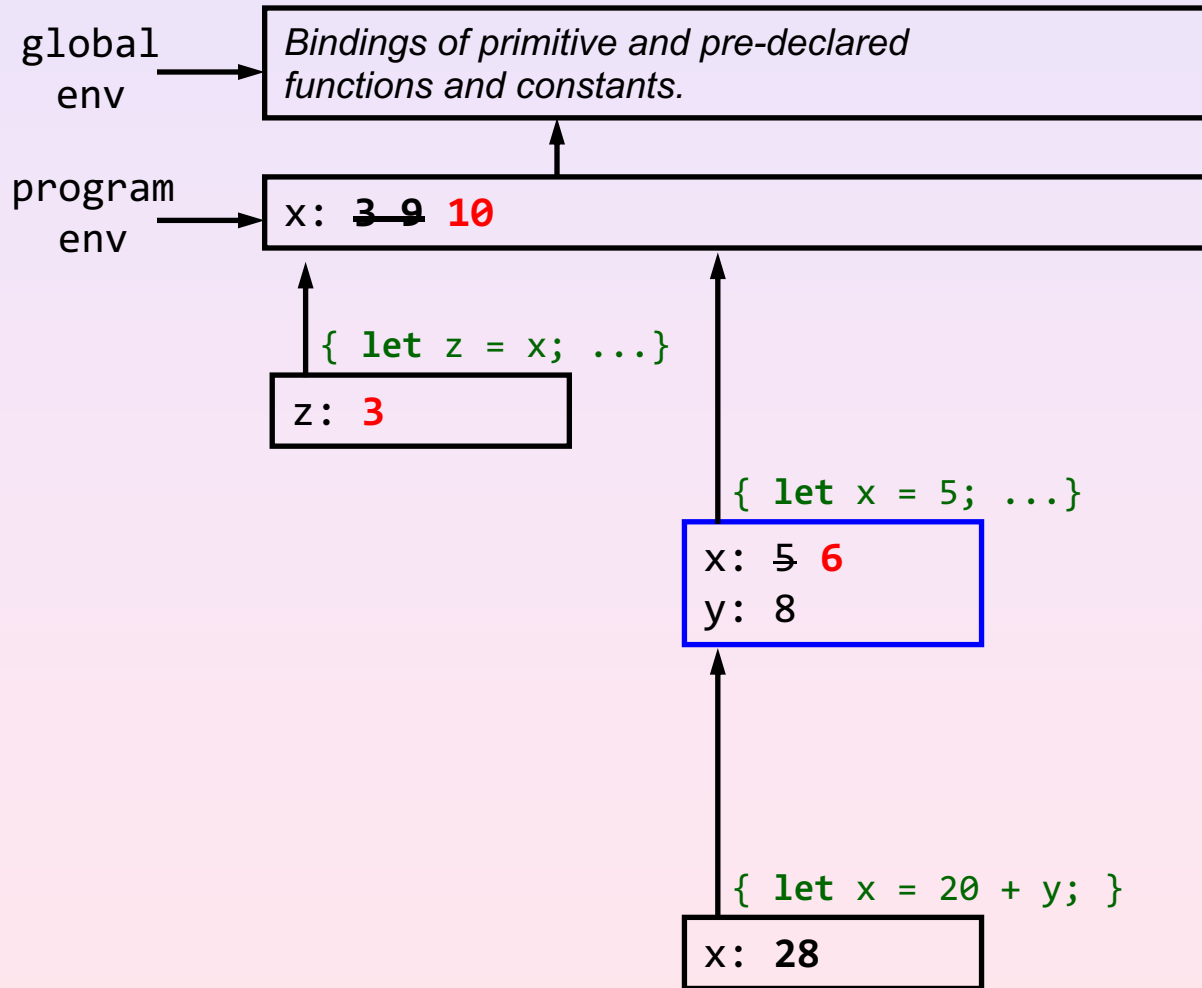
```

let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;
    
```



```

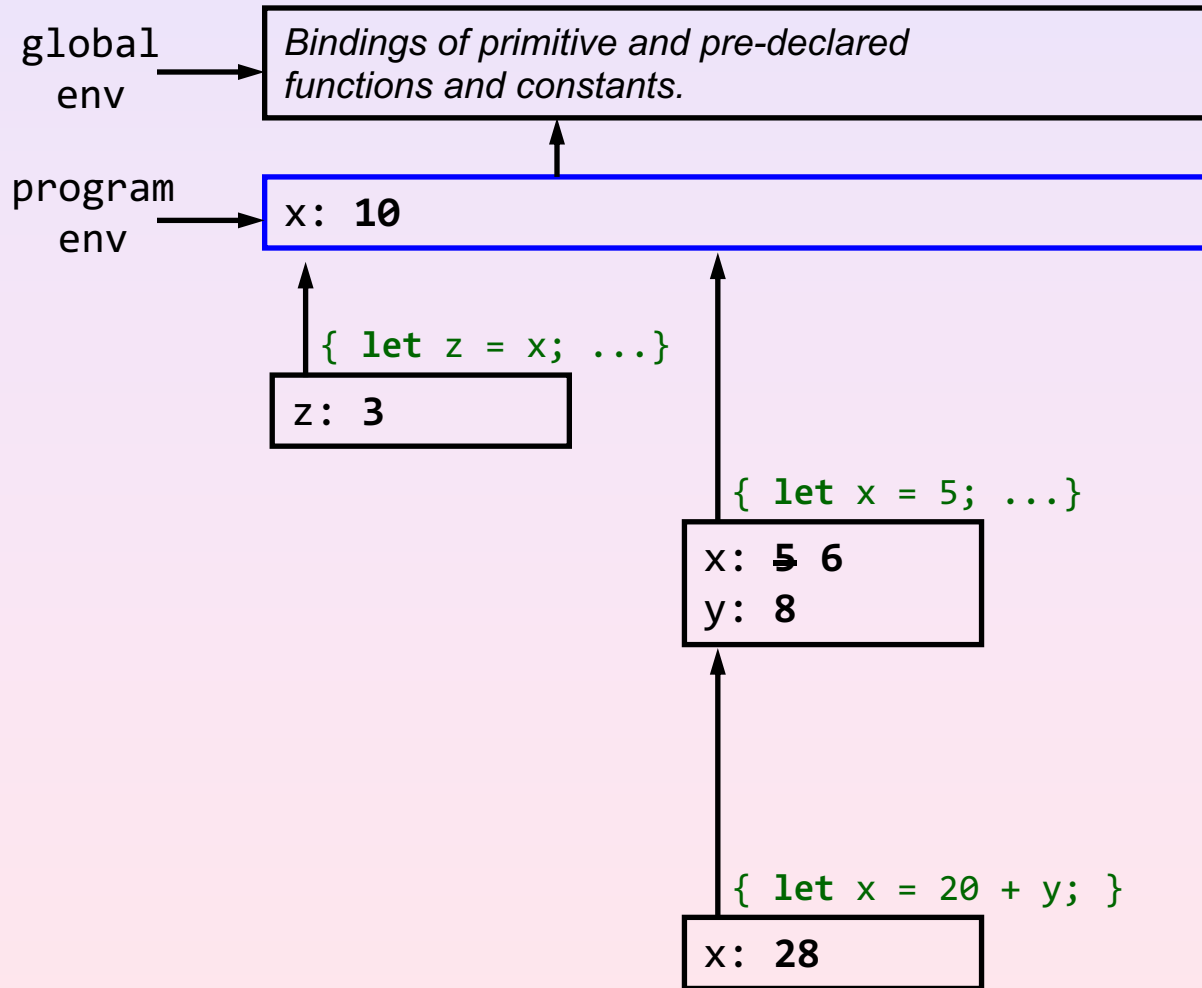
let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

x = x + 1;

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}
x;

```

```

let x = 3;

if (x <= 1) {
    let y = x;
    x = y + 1;
} else {
    let z = x;
    x = z * z;
}

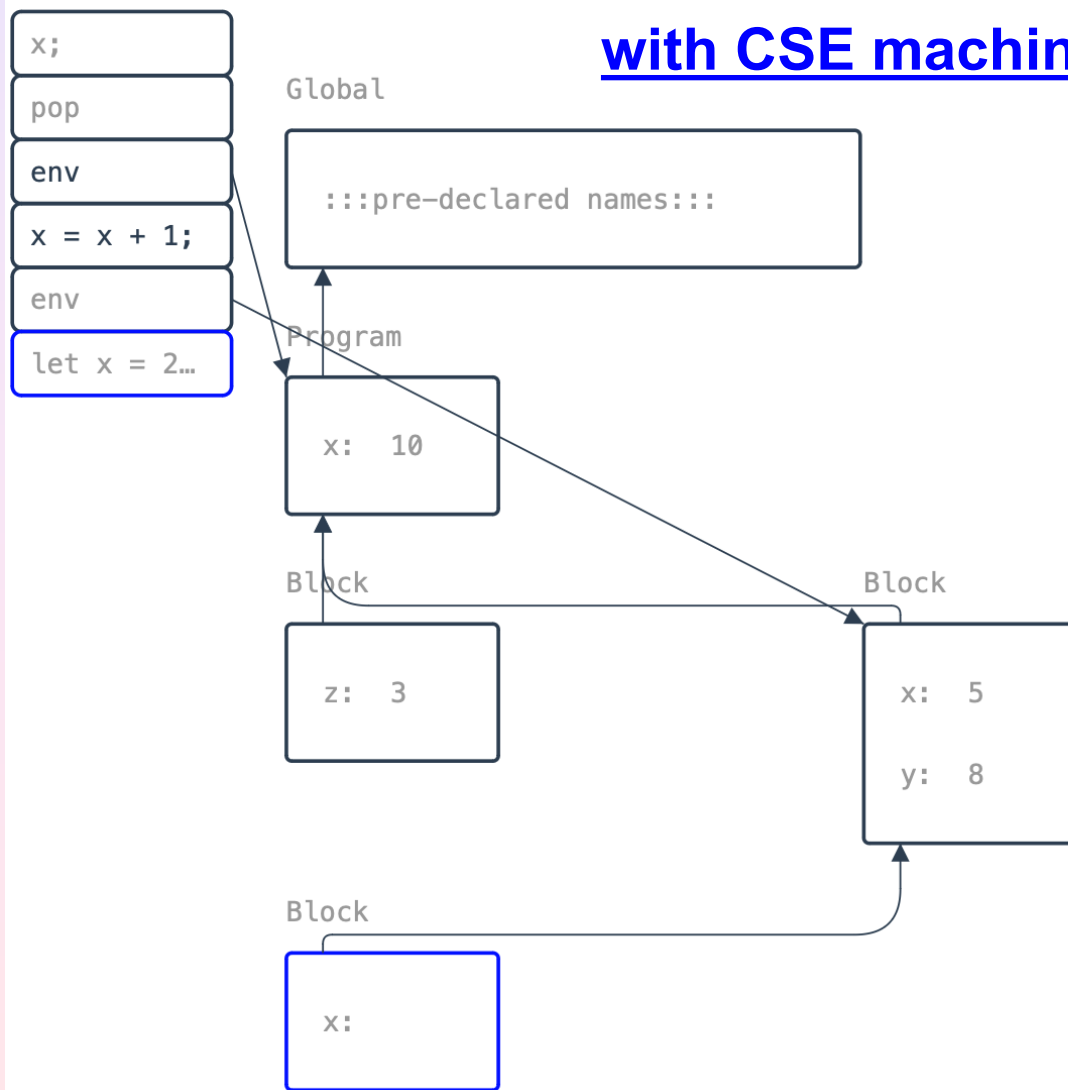
{
    x = x + 1;
}

{
    let x = 5;
    let y = 8;
    {
        let x = 20 + y;
    }
    x = x + 1;
}

x;

```

with CSE machine



```
let x = 3;

if (x <= 1) {
  let y = x;
  x = y + 1;
} else {
  let z = x;
  x = z * z;
}

x = x + 1;

let x = 5;
let y = 8;
{
  let x = 20 + y;
}
x = x + 1;
x;
```

Outline

- Recap: CSE Machine
- Blocks
- Functions
- Stateful Data and Identity

How will functions work in CSE machine?

```
function make_adder(x) {  
    return y => x + y;  
}  
const add_4 =  
    make_adder(4);  
add_4(5);
```

[Show in
Playground](#)

In CSE machine: How does
add_4 function know
that **x** is **4**?

The substitution model
replaces **x** by **4** in the
body of **add_4**.

Evaluation of function applications

- Every **function application** extends the environment in which the *function was created*
- The **new frame** contains bindings of **parameters** to arguments
 - New **frame is not created** if the function has **no parameters**
- The **function body block** is evaluated in this new environment

Evaluation of lambda expressions

- Lambda expression:

$(var1, var2, \dots) \Rightarrow \{ body \}$

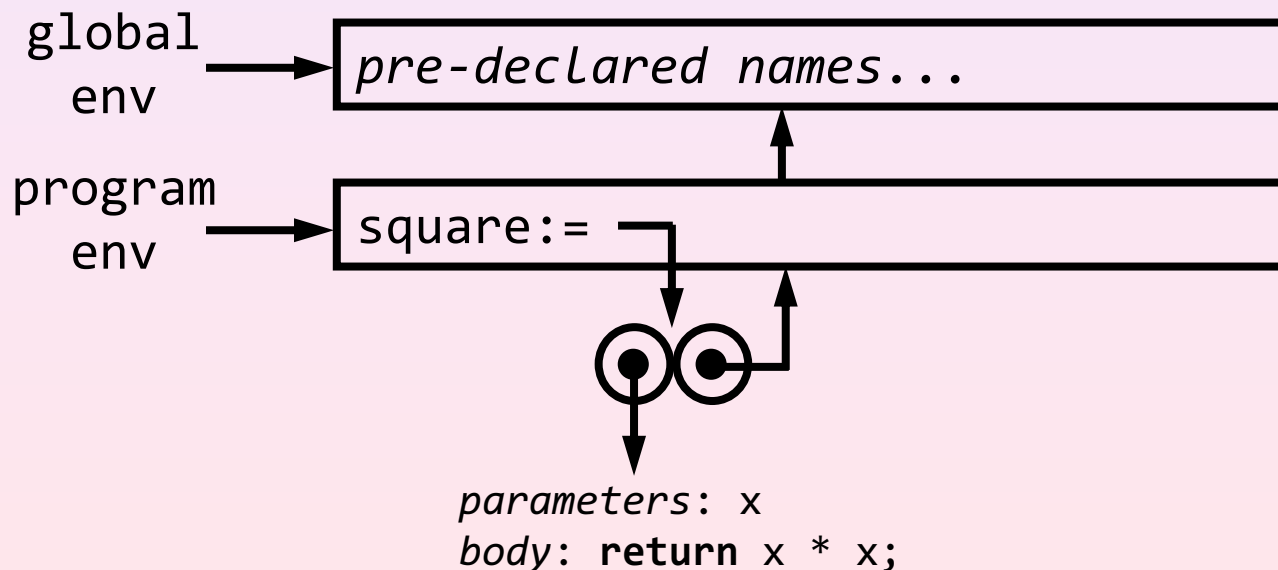
- Creates a **function object**, represented as **two circles**
- One points to *body* (text)
- The other points to the environment **in which the lambda expression was evaluated**
- A function therefore remembers the **environment in which it was created**

Example for evaluating lambda expressions

```
const square = x => x * x;  
square(5);
```

[Show in
Playground](#)

- After evaluating the constant declaration in **program environment**:



Example for evaluating function declarations

```
function square(x) { return x * x; }
```

is equivalent to*

```
const square = x => x * x;
```


Evaluation of function applications

- To evaluate a **function application**:

fun(arg1, arg2, ..., argn);

- Evaluate the **subexpressions** in the **current environment**
- Apply the value of the **function subexpression** (*fun*) to the values of the **argument subexpressions** (*arg1, arg2, ..., argn*)

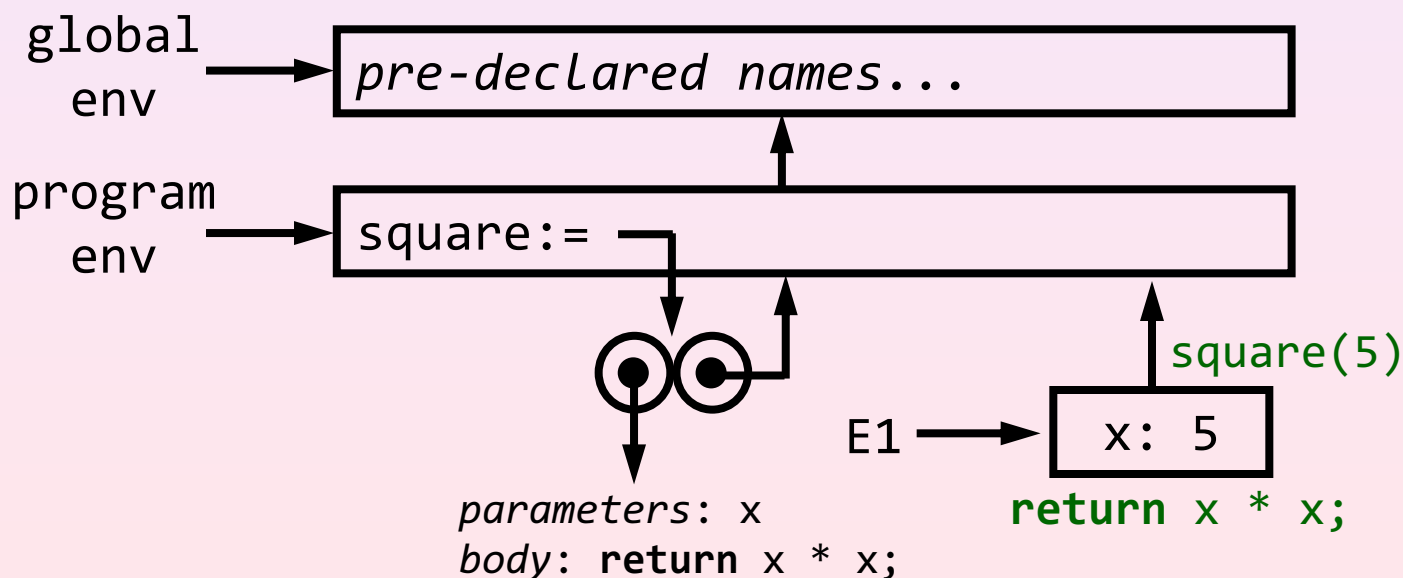
Applying a function value to argument values

- Create a new frame that **points to the environment of the function**
 - **The environment of the function** is the environment in which the function was created
- In this new frame, **bind the parameters to the arguments**
- **Parameters are variables:** assignment allowed
- Evaluate the **body block** of the function in the new environment

Example: `square(5);`

- Evaluate subexpressions `square` and `5` in **program env**
- Create new frame that points to environment of `square`
- In this frame, bind value `5` to parameter `x`
- Evaluate body `return x * x;`

[Show in
Playground](#)



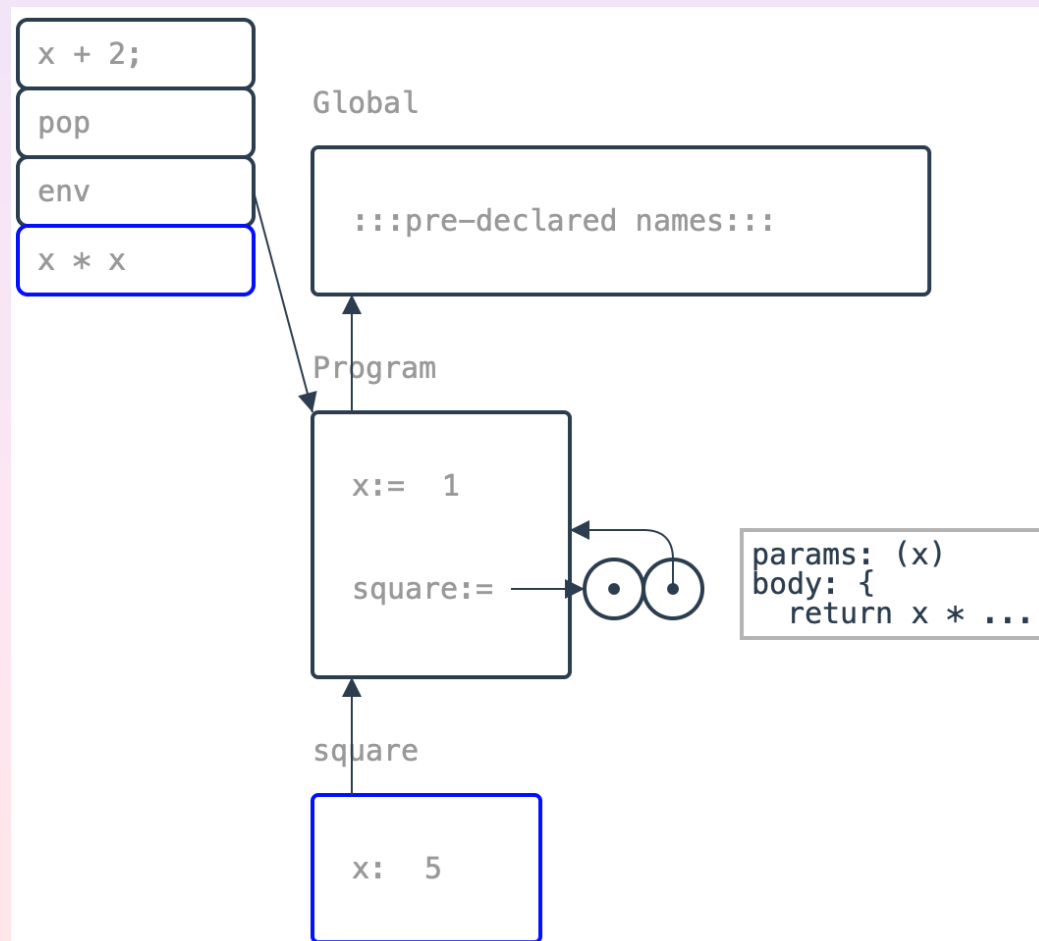
Another small difficulty

[Show in
Playground](#)

```
const x = 1;  
const square =  
  x => x * x;  
square(5);  
x + 2;
```

Problem: Old environment
is still needed after function
returns

Solution: Use the same
technique as with blocks:
env instructions



Example with local declaration

- The `cube` function has a **local constant** `y`

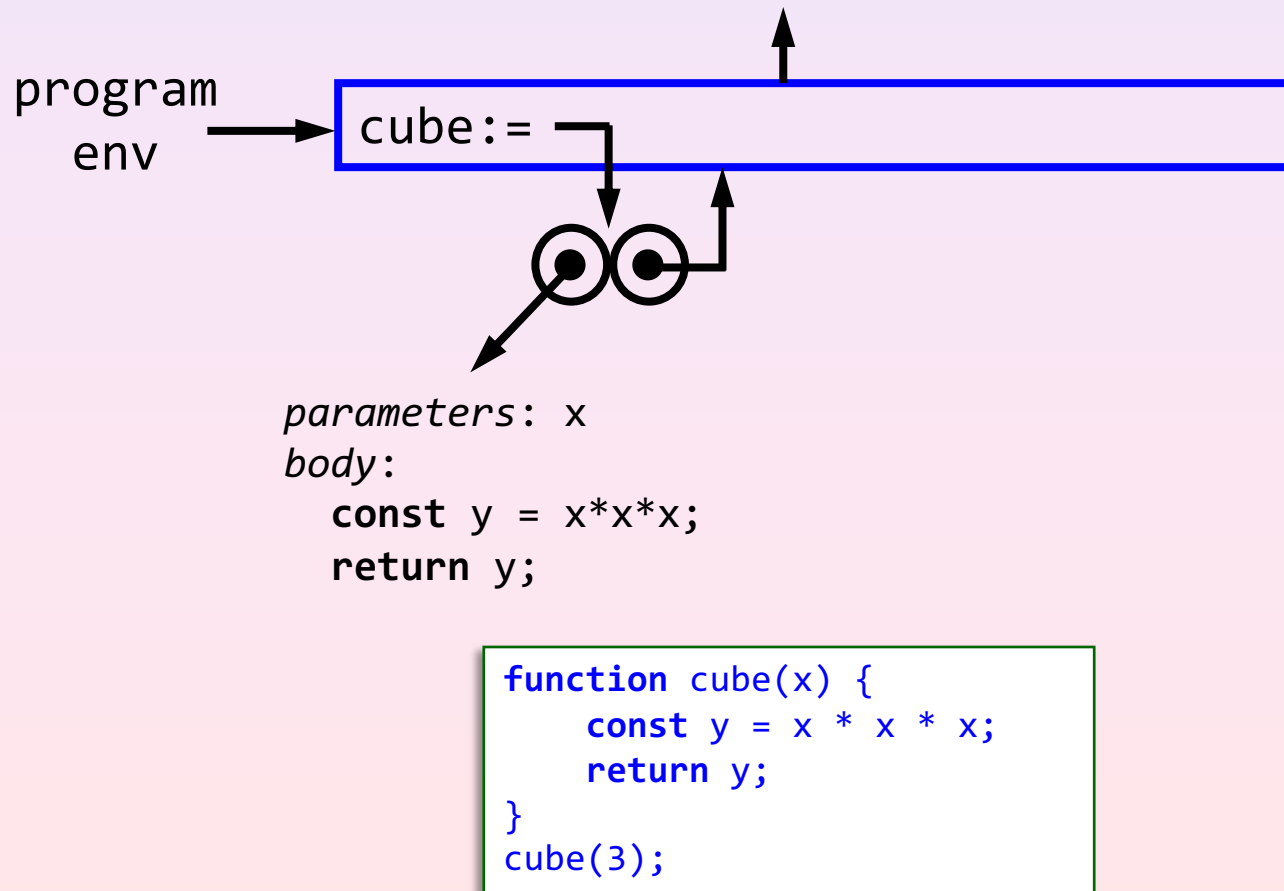
[Show in
Playground](#)

```
function cube(x) {  
    const y = x * x * x;  
    return y;  
}  
cube(3); ➔ 27
```

cube example

- After evaluating the declaration of function `cube`

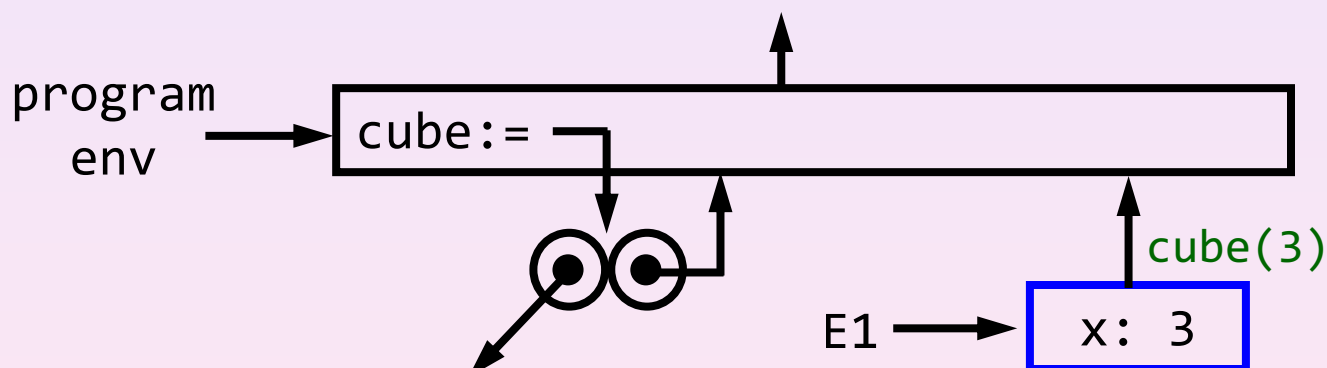
[Show in
Playground](#)



cube example

- Evaluate `cube(3)` in **program env**

[Show in
Playground](#)



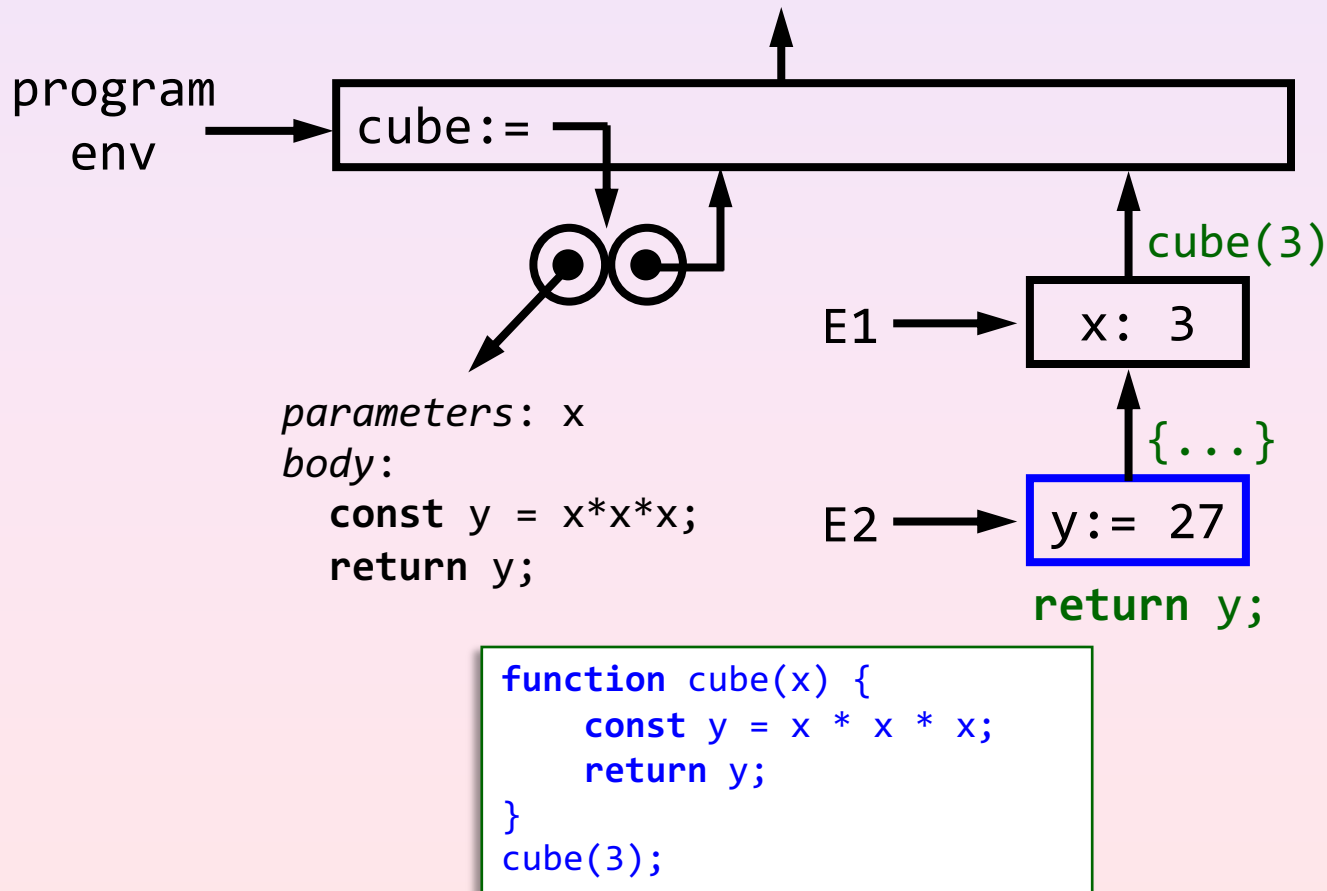
parameters: x
body:
const y = x*x*x;
return y;

```
function cube(x) {  
  const y = x * x * x;  
  return y;  
}  
cube(3);
```

cube example

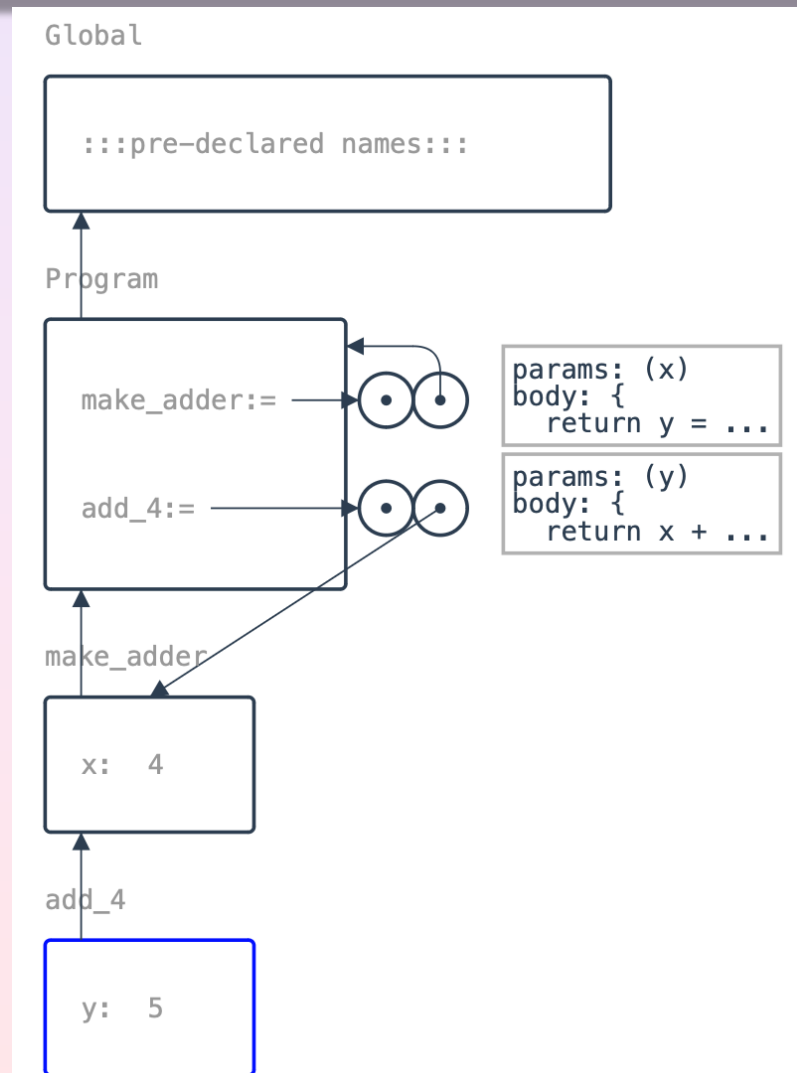
- Evaluate function **body block**

[Show in
Playground](#)



Revisiting make_adder example

```
function make_adder(x) {  
  return y => x + y;  
}  
  
const add_4 =  
  make_adder(4);  
  
add_4(5);
```



[Show in Playground](#)

Example with recursion

- Recursive `factorial` function

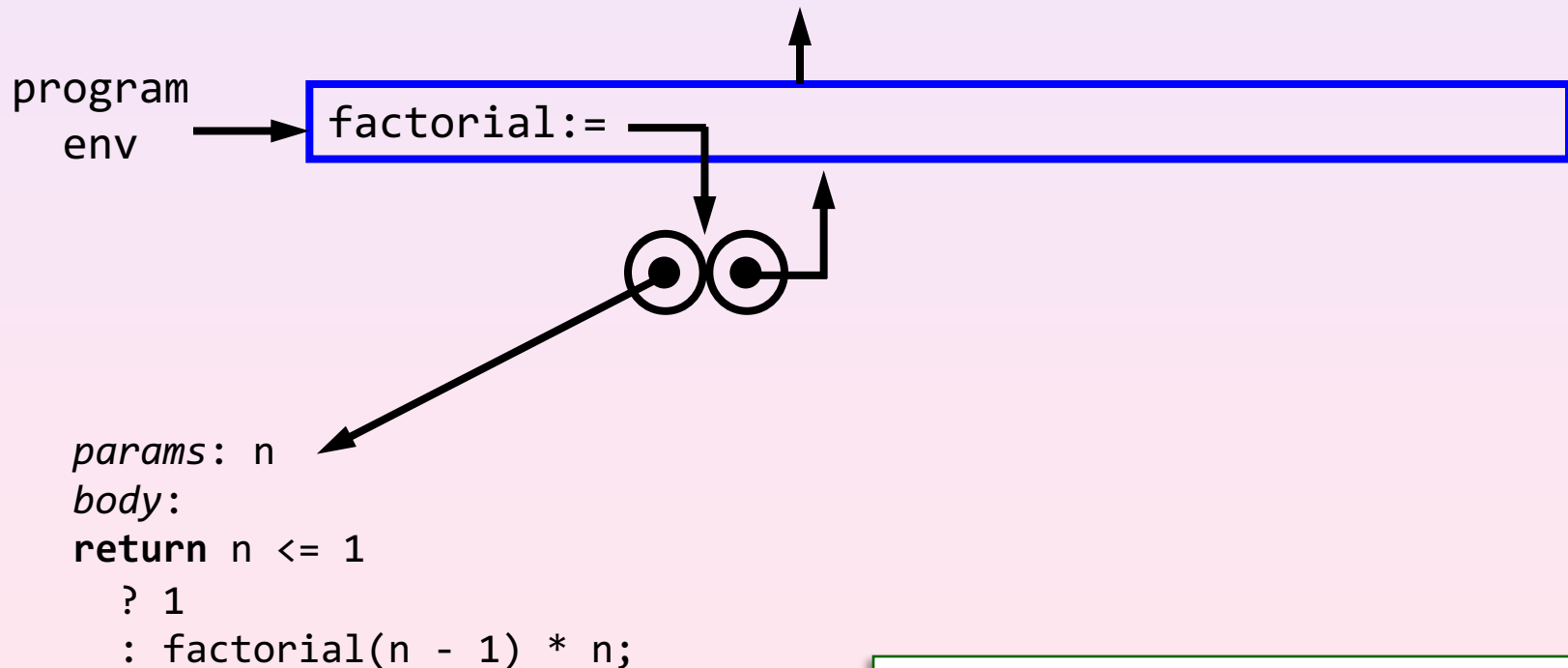
[Show in
Playground](#)

```
function factorial(n) {  
  return n <= 1  
    ? 1  
    : factorial(n - 1) * n;  
}  
factorial(2); ➔ 2
```

factorial example

- After evaluating the declaration of function `factorial`

[Show in
Playground](#)

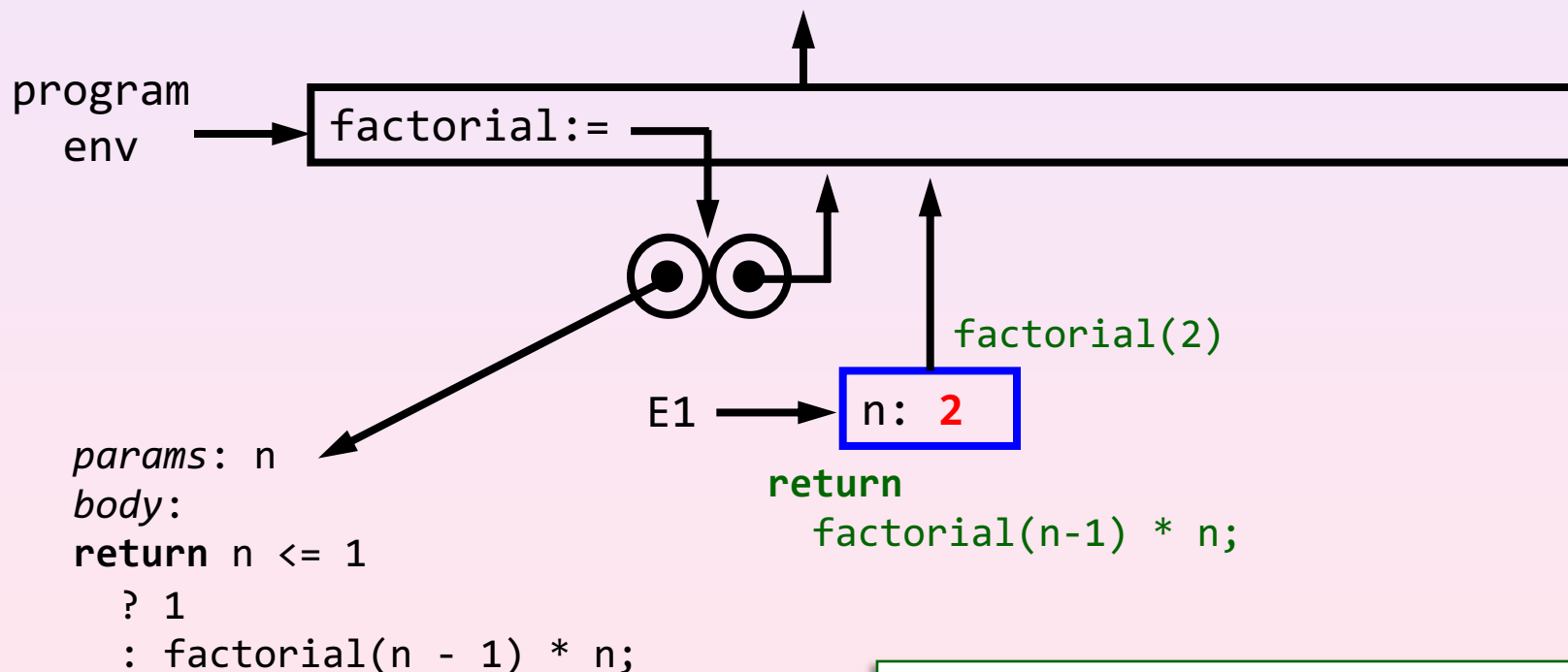


```
function factorial(n) {  
    return n <= 1 ? 1 : factorial(n - 1) * n;  
}  
factorial(2);
```

factorial example

- Evaluate `factorial(2)` in **program environment**

[Show in
Playground](#)

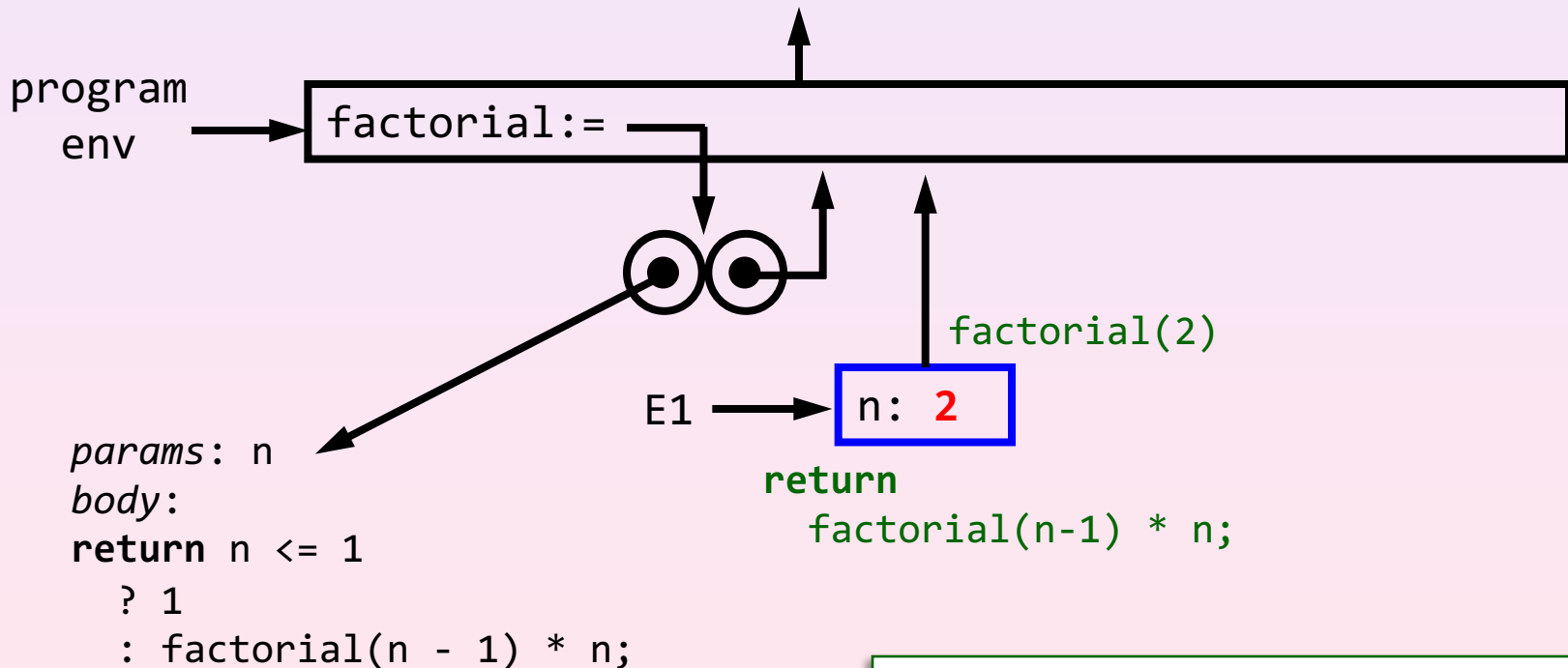


```
function factorial(n) {  
    return n <= 1 ? 1 : factorial(n - 1) * n;  
}  
factorial(2);
```

factorial example

- Evaluate `factorial(n-1)` in **environment E1**

[Show in Playground](#)

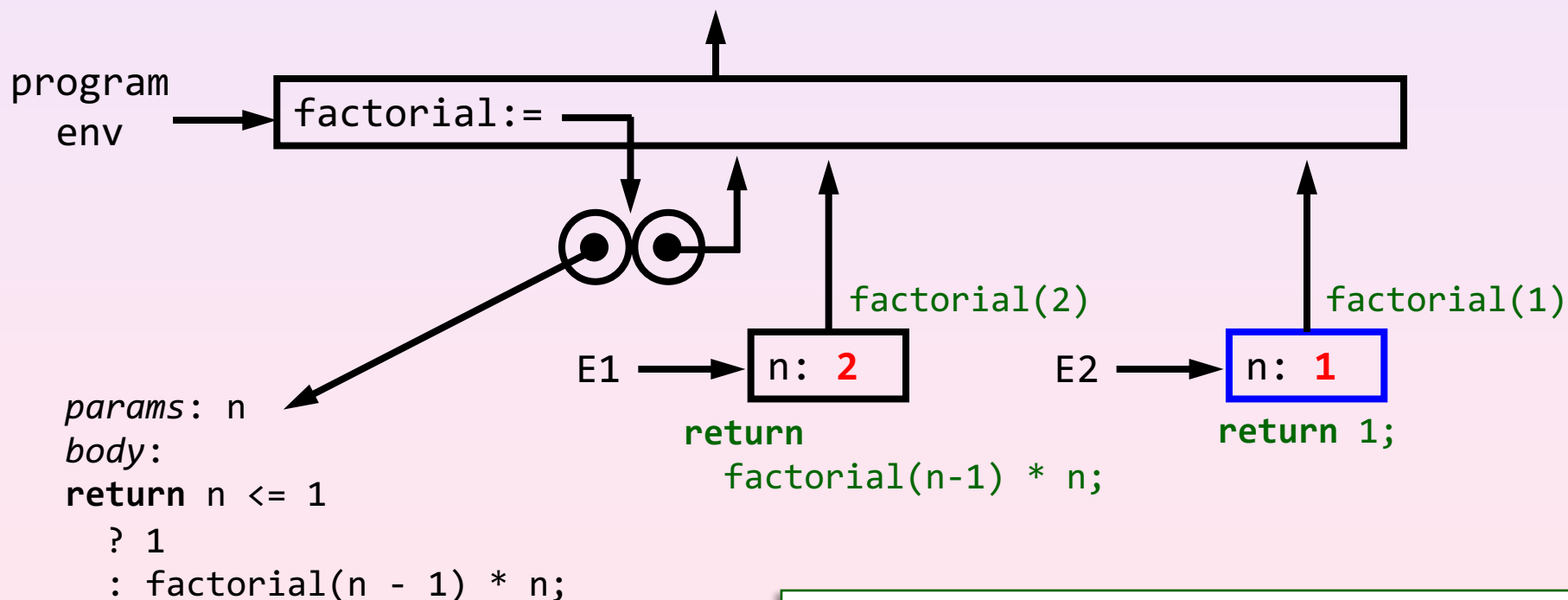


```
function factorial(n) {  
    return n <= 1 ? 1 : factorial(n - 1) * n;  
}  
factorial(2);
```

factorial example

- Evaluate **return 1;** in **environment E2**

[Show in Playground](#)

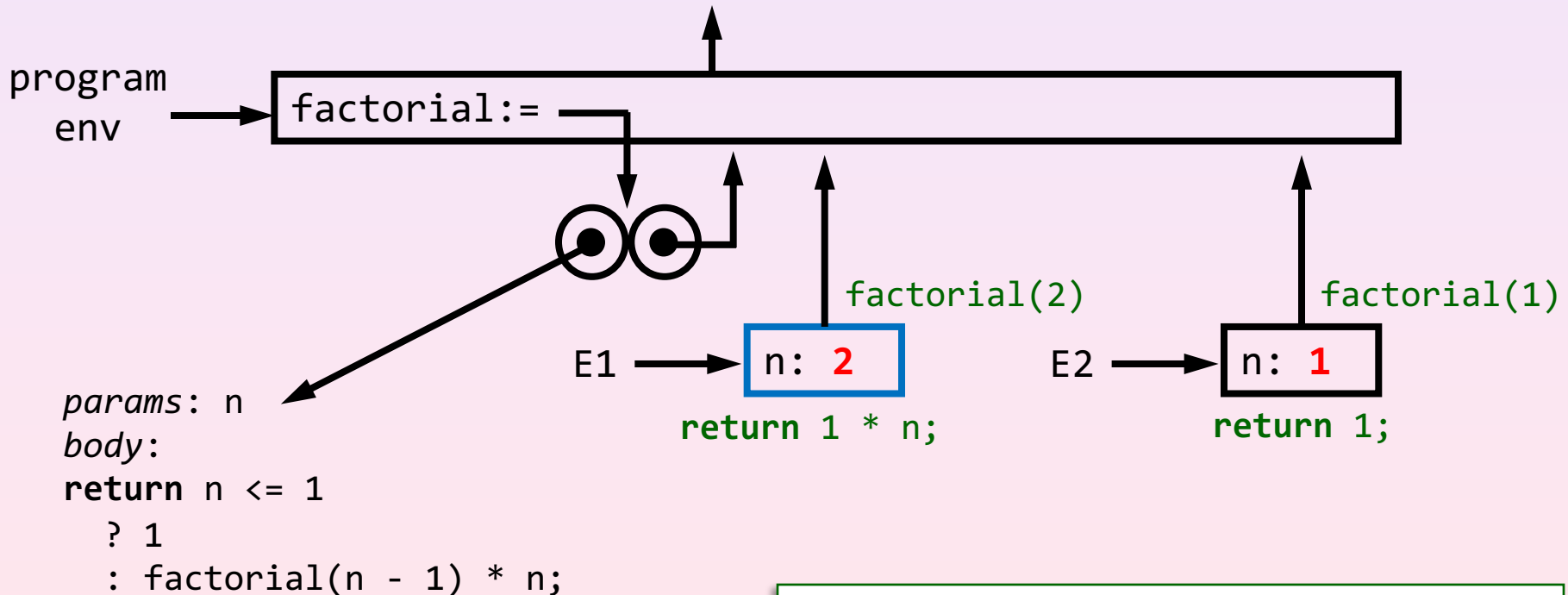


```
function factorial(n) {  
    return n <= 1 ? 1 : factorial(n - 1) * n;  
}  
factorial(2);
```

factorial example

- Evaluate **return** $n * 1$; in **environment** E1

[Show in Playground](#)



```
function factorial(n) {  
    return n <= 1 ? 1 : factorial(n - 1) * n;  
}  
factorial(2);
```

make_withdraw example

[Show in
Playground](#)

```
function make_withdraw(balance) {  
  return amount => {  
    if (balance >= amount) {  
      balance = balance - amount;  
      return balance;  
    } else {  
      return "Insufficient funds";  
    }  
  };  
}  
  
const w1 = make_withdraw(100);  
w1(40); ➔ 60
```


make_withdraw example

- After declaring function `make_withdraw`

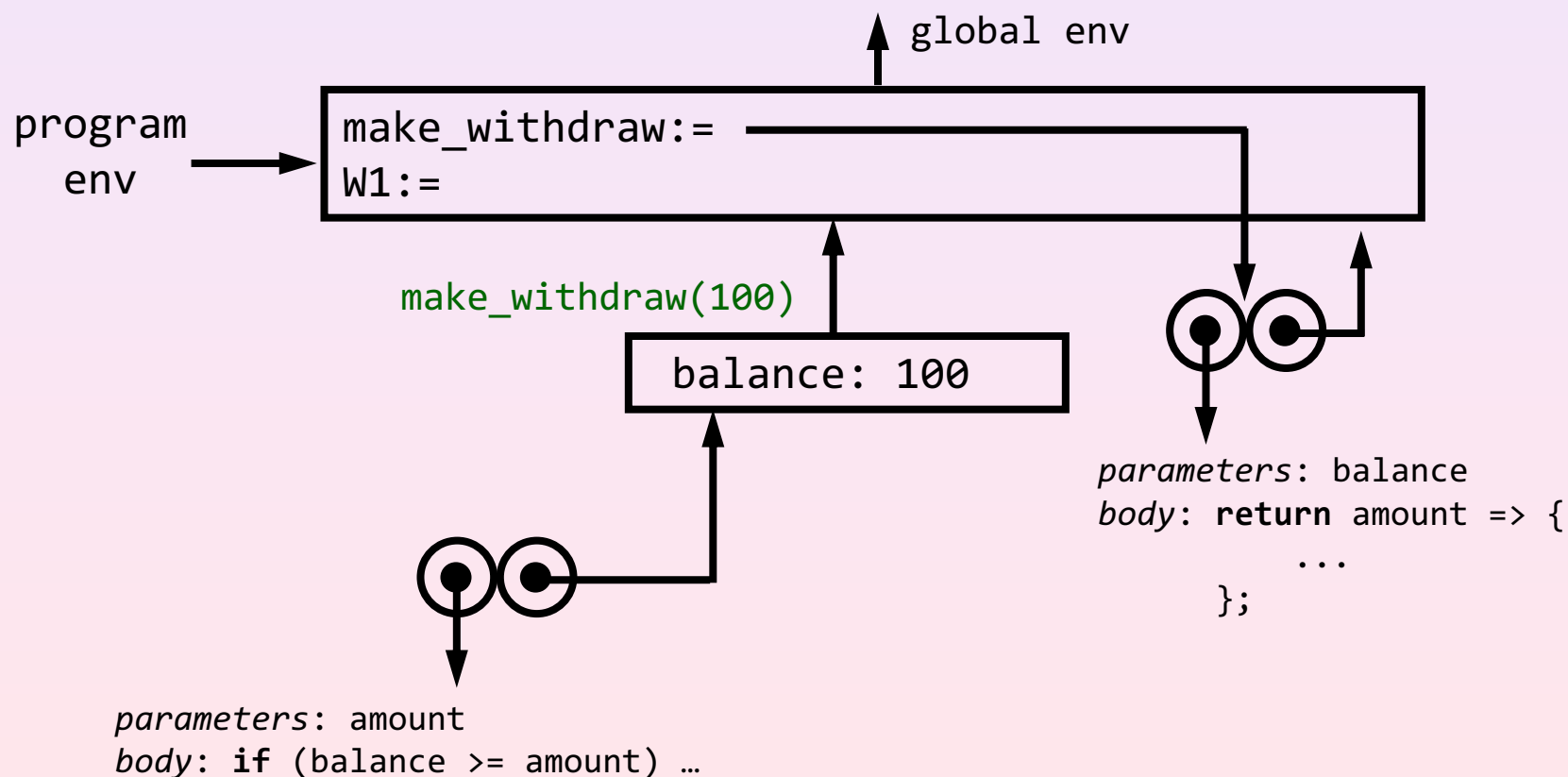
[Show in
Playground](#)



make_withdraw example

- Evaluate `make_withdraw(100)`

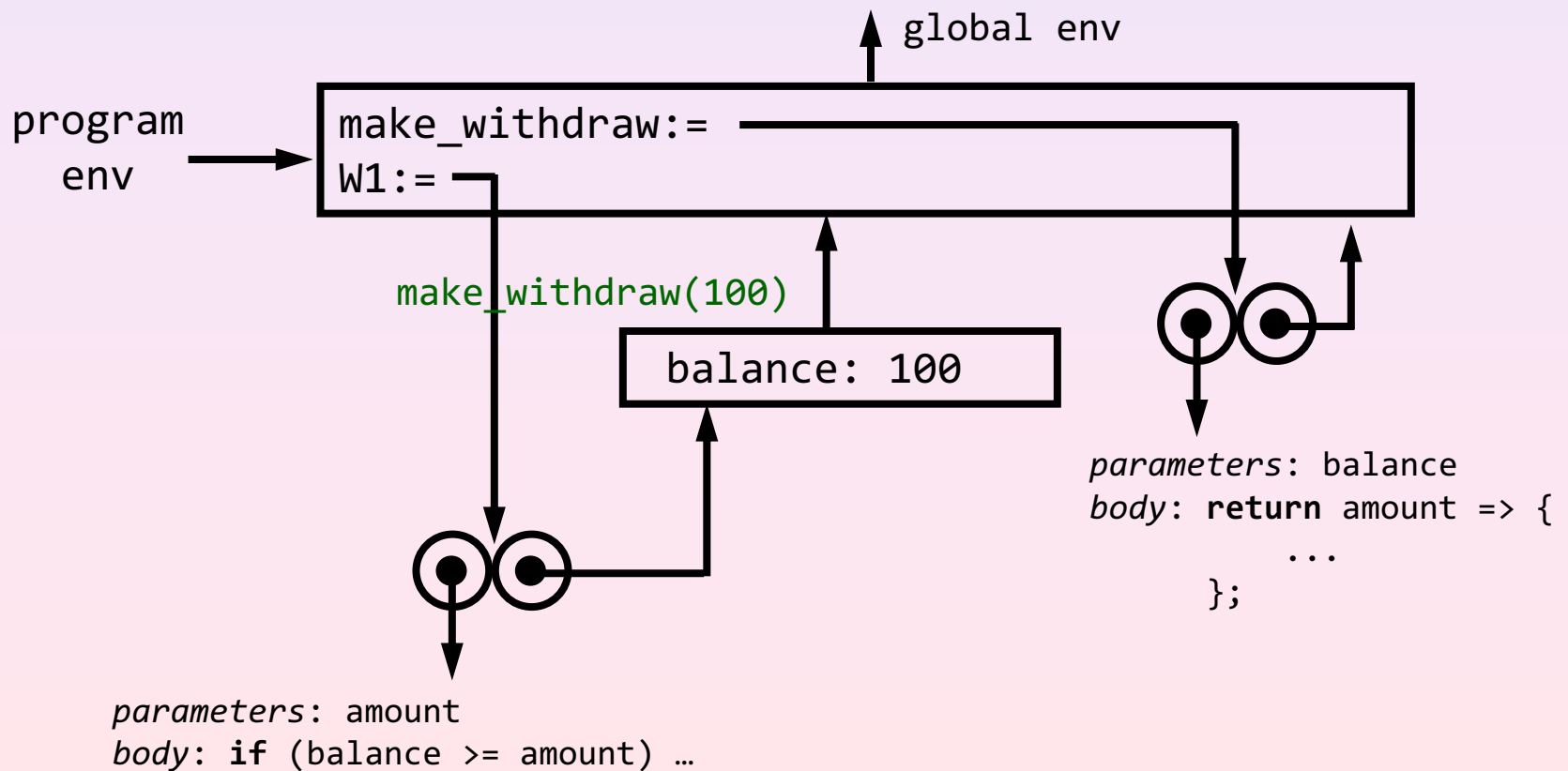
[Show in
Playground](#)



make_withdraw example

- Evaluate `const W1 = make_withdraw(100);`

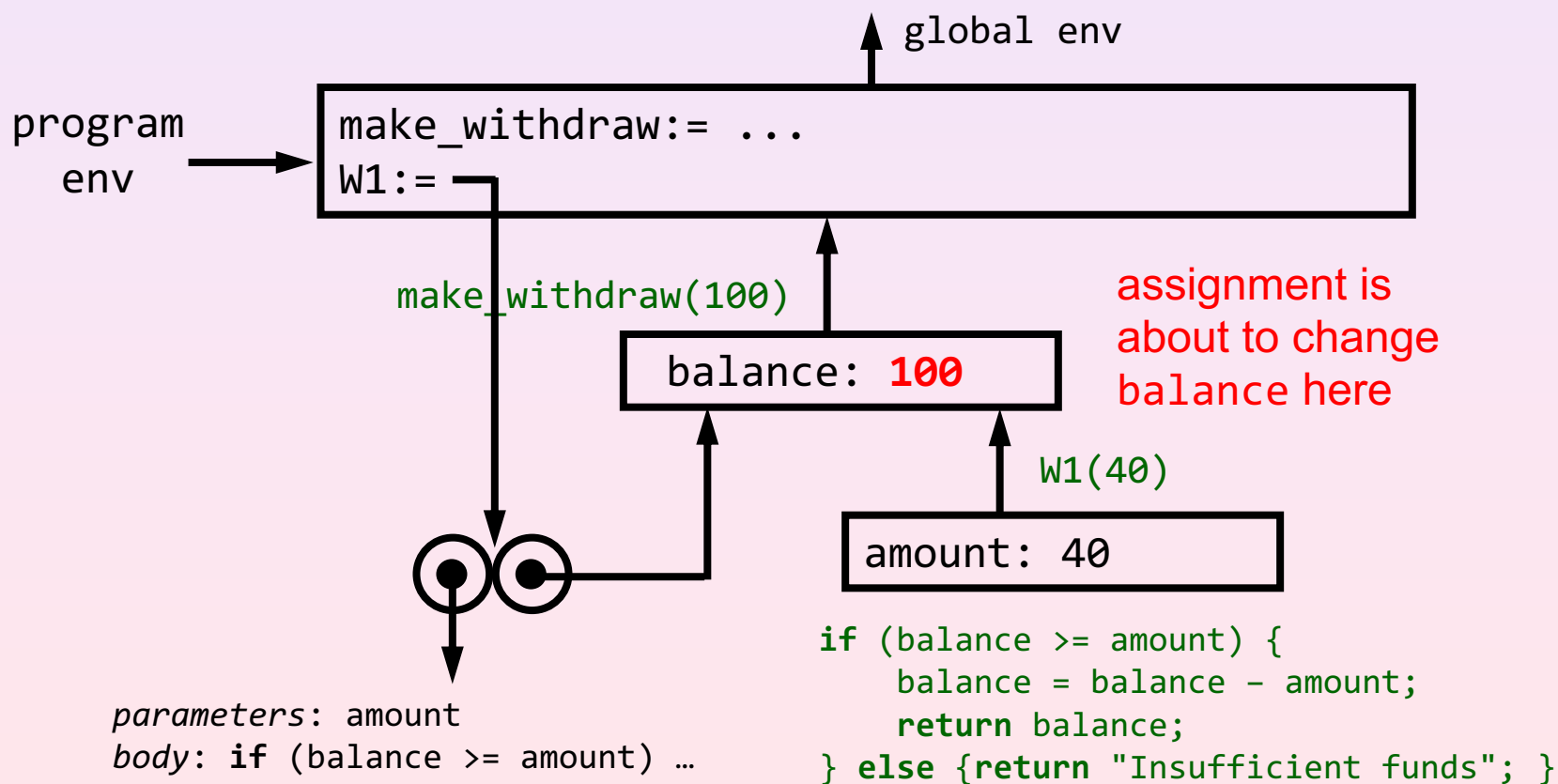
[Show in
Playground](#)



make_withdraw example

- Evaluate `W1(40)`

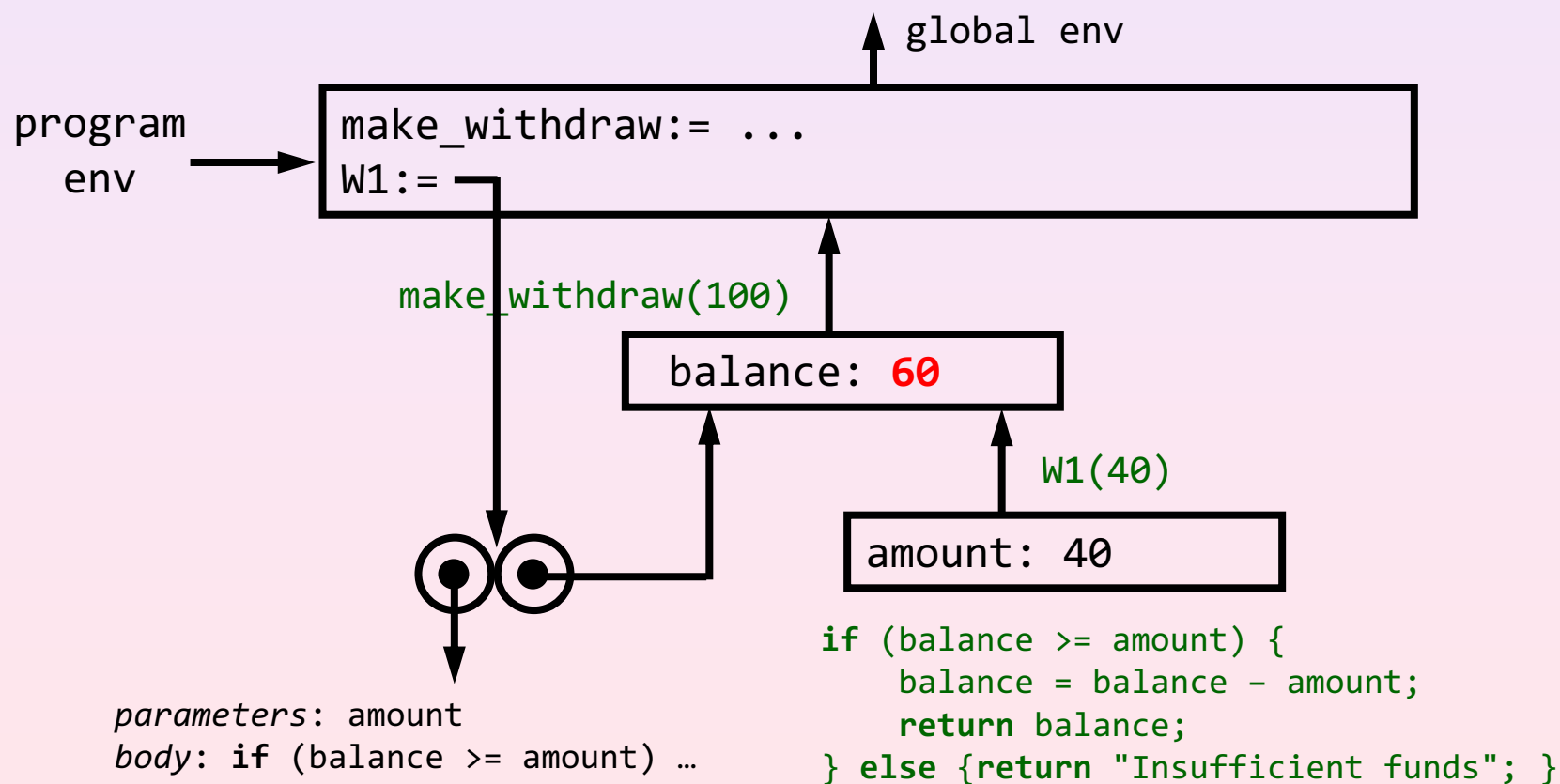
[Show in
Playground](#)



make_withdraw example

- Evaluate `W1(40)`

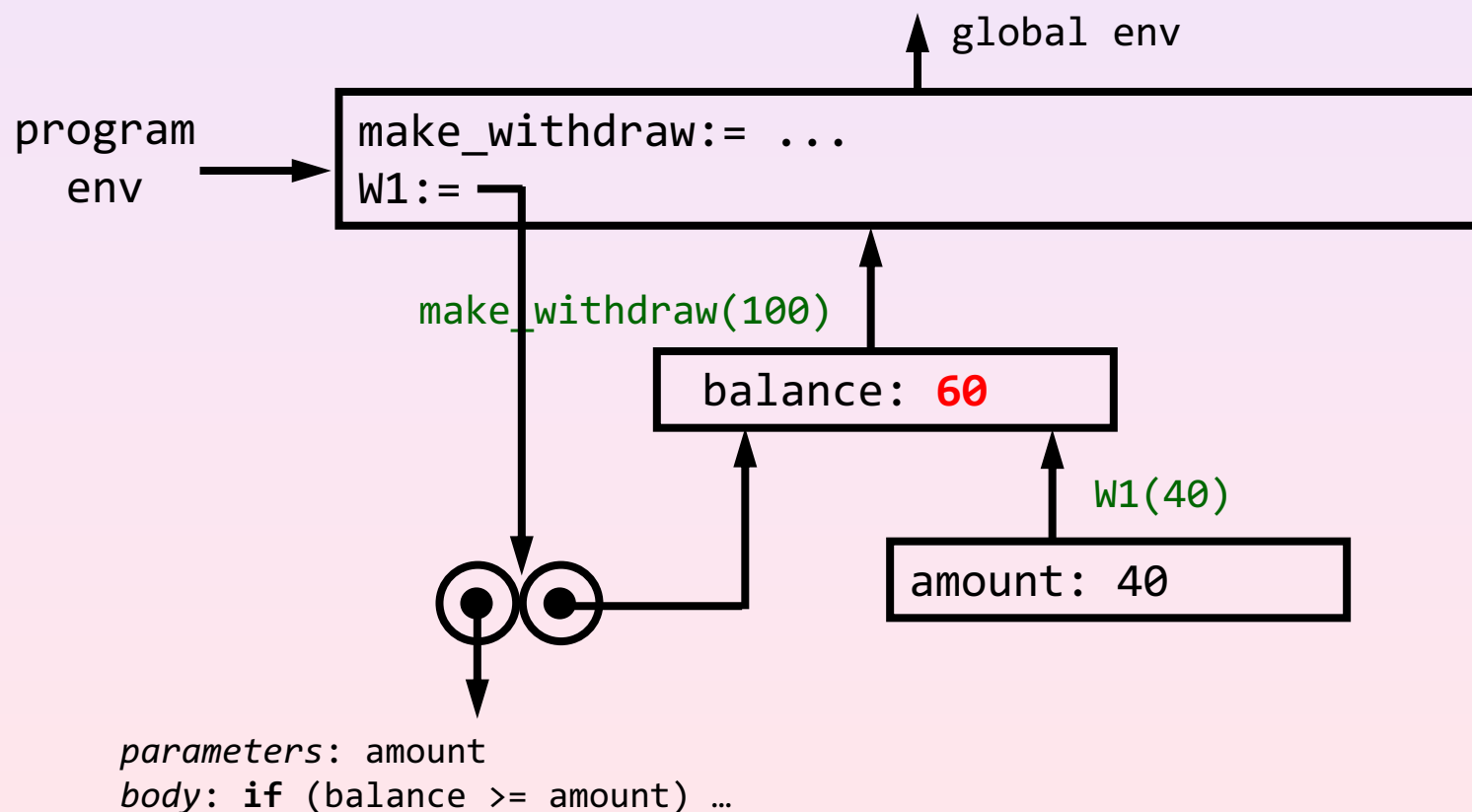
[Show in
Playground](#)



make_withdraw example

- Environment after evaluating `W1(40);`

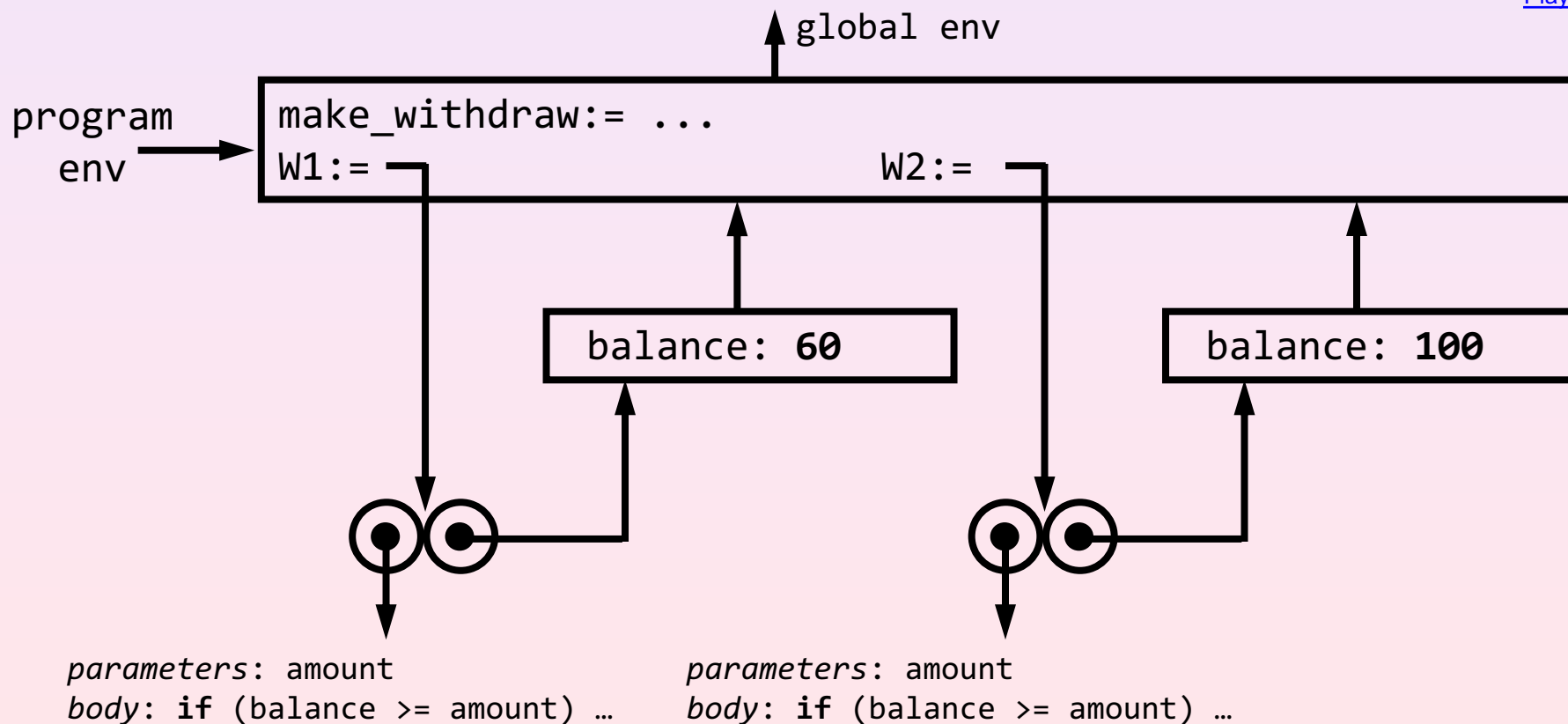
[Show in
Playground](#)



make_withdraw example

- Environment after evaluating `const W2 = make_withdraw(100);`
- `W2` has its own state variable

[Show in
Playground](#)



A third difficulty: “premature” return

```
function abs(x) {  
  if (x >= 0) {  
    return x;  
  }  
  return -x;  
}  
abs(5);
```

[Show in
Playground](#)

Problem: In JavaScript and Source §3, return can be anywhere in body. What if there are still items on control when function returns?

Solution: Place a mark on control and return to mark

update example

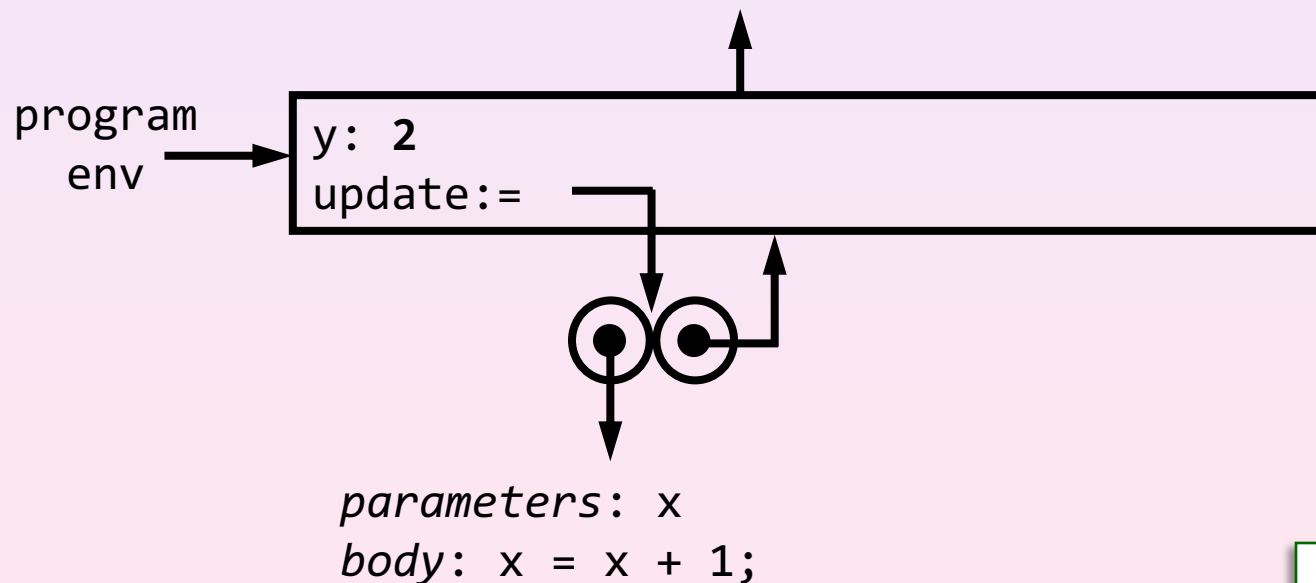
What is the result of this program?

```
function update(x) {  
    x = x + 1;  
}  
let y = 2;  
update(y);  
y; ➔ 2
```

[Show in
Playground](#)

update example

After evaluating declarations of function `update` and variable `y`

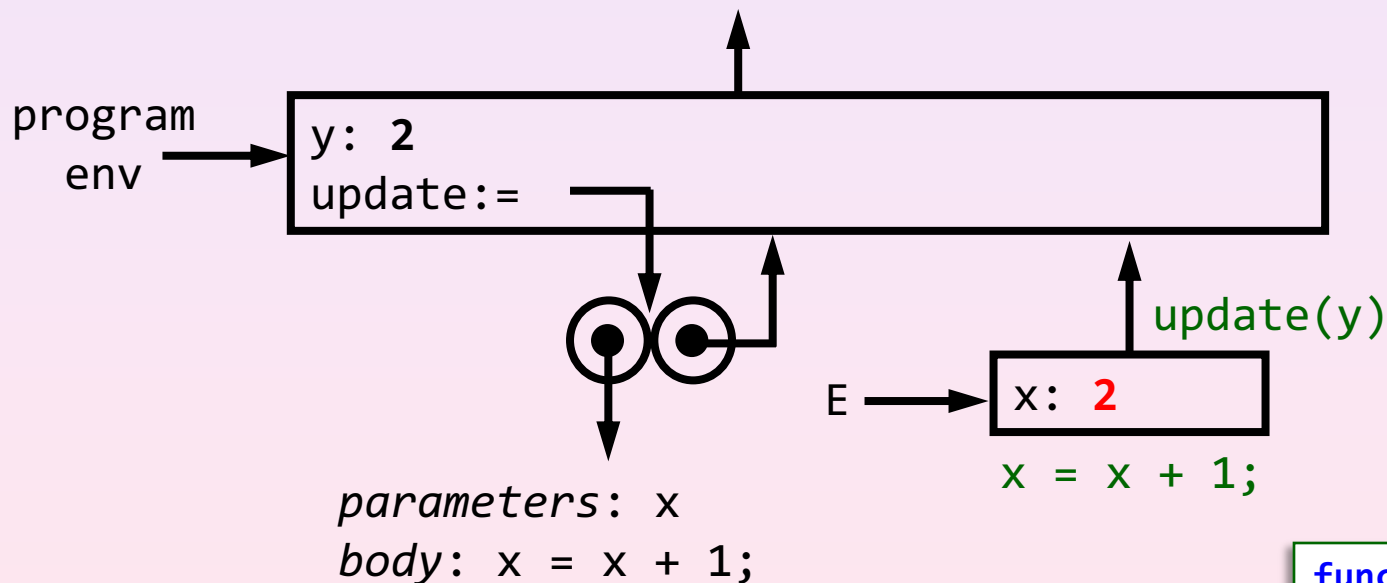


[Show in
Playground](#)

```
function update(x) {  
    x = x + 1;  
}  
let y = 2;  
update(y);  
y;
```

update example

Evaluate `update(y)` in **program env**

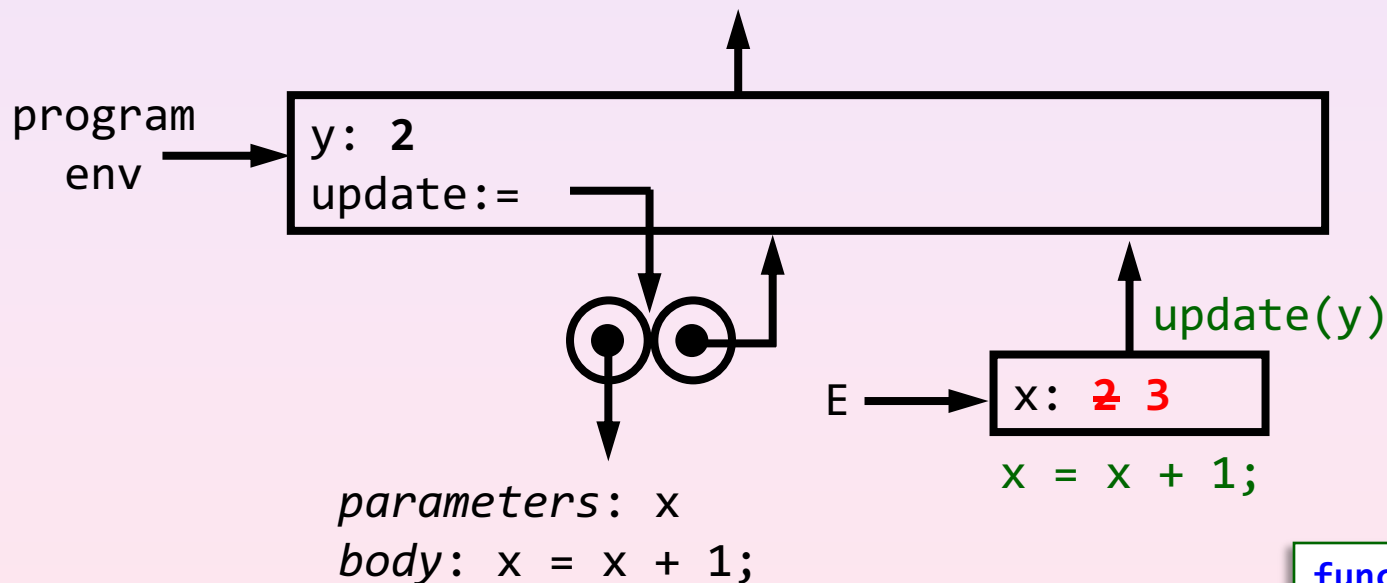


[Show in Playground](#)

```
function update(x) {  
    x = x + 1;  
}  
let y = 2;  
update(y);  
y;
```

update example

Evaluate $x = x + 1$; in Environment E

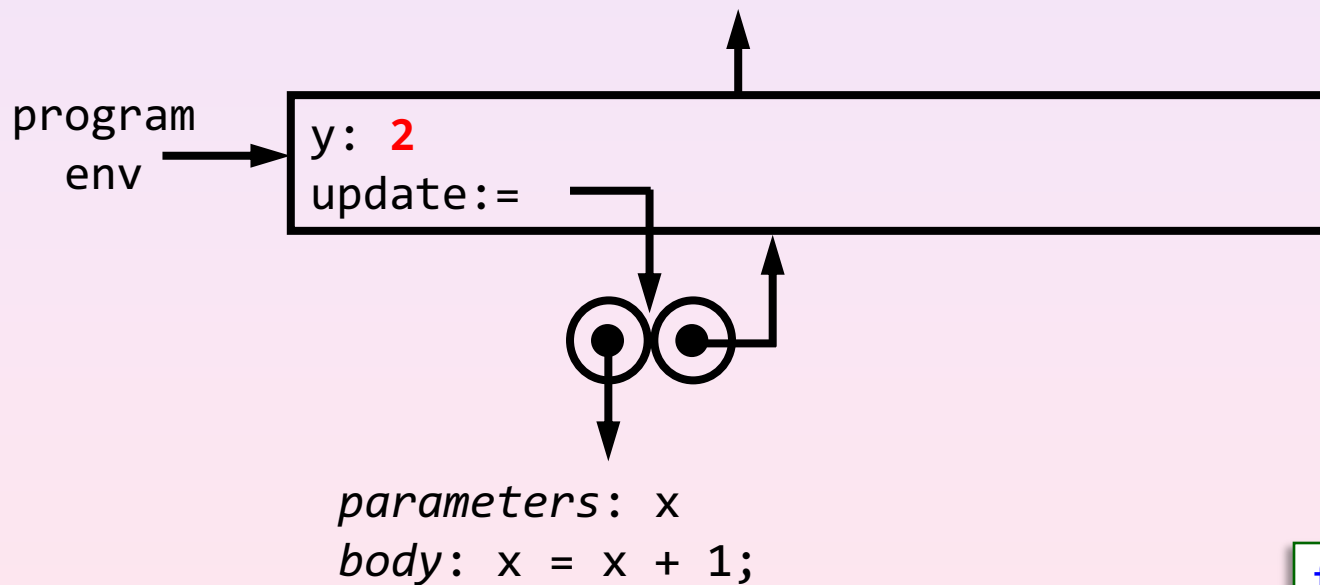


[Show in Playground](#)

```
function update(x) {  
    x = x + 1;  
}  
let y = 2;  
update(y);  
y;
```

update example

Evaluate *y* in program env



[Show in
Playground](#)

```
function update(x) {  
    x = x + 1;  
}  
let y = 2;  
update(y);  
y;
```

Outline

- Recap: CSE Machine
- Blocks
- Functions
- Stateful Data and Identity

pair_add_one example

What is the result of this program?

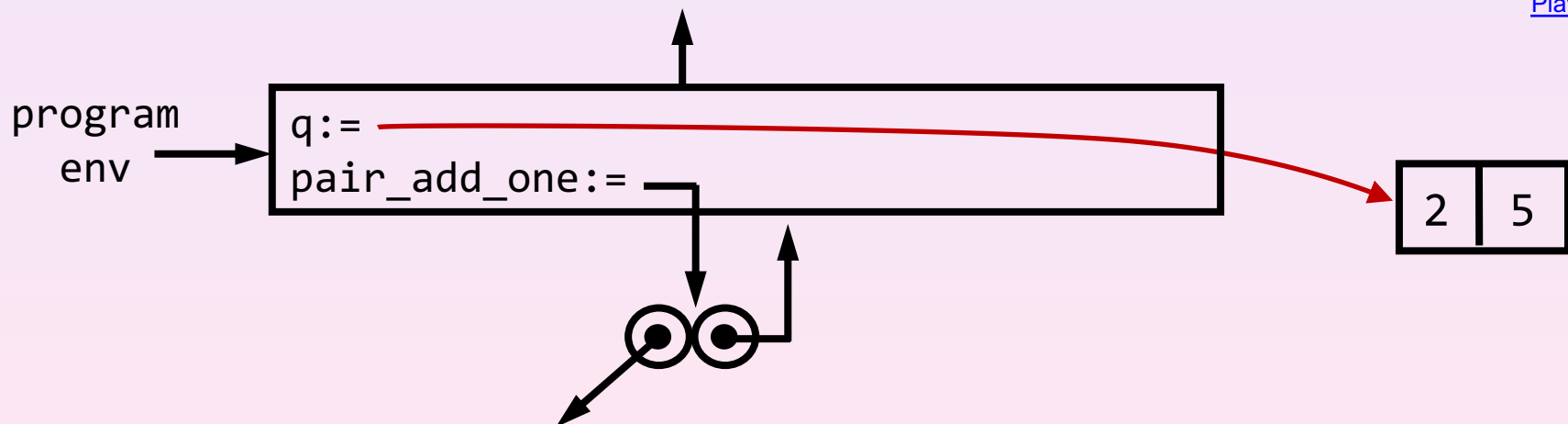
[Show in
Playground](#)

```
function pair_add_one(p) {  
    set_head(p, head(p) + 1);  
    set_tail(p, tail(p) + 1);  
}  
const q = pair(2, 5);  
pair_add_one(q);  
q; ➔ [3, 6]
```

pair_add_one example

- After evaluating declarations of function `pair_add_one` and constant `q`

[Show in
Playground](#)



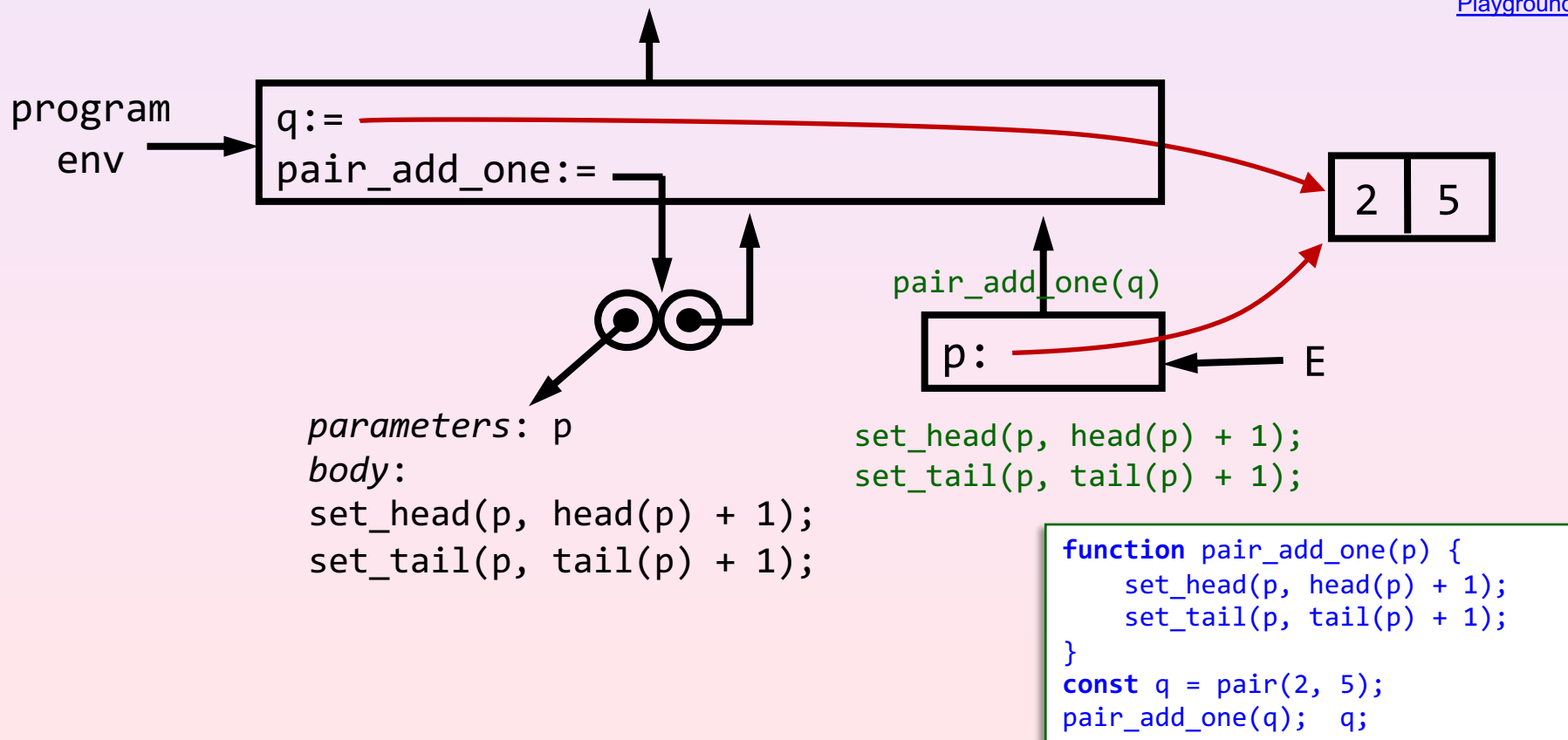
parameters: p
body:
set_head(p, head(p) + 1);
set_tail(p, tail(p) + 1);

```
function pair_add_one(p) {  
    set_head(p, head(p) + 1);  
    set_tail(p, tail(p) + 1);  
}  
const q = pair(2, 5);  
pair_add_one(q); q;
```


pair_add_one example

- Evaluate `pair_add_one(q)` in **program env**

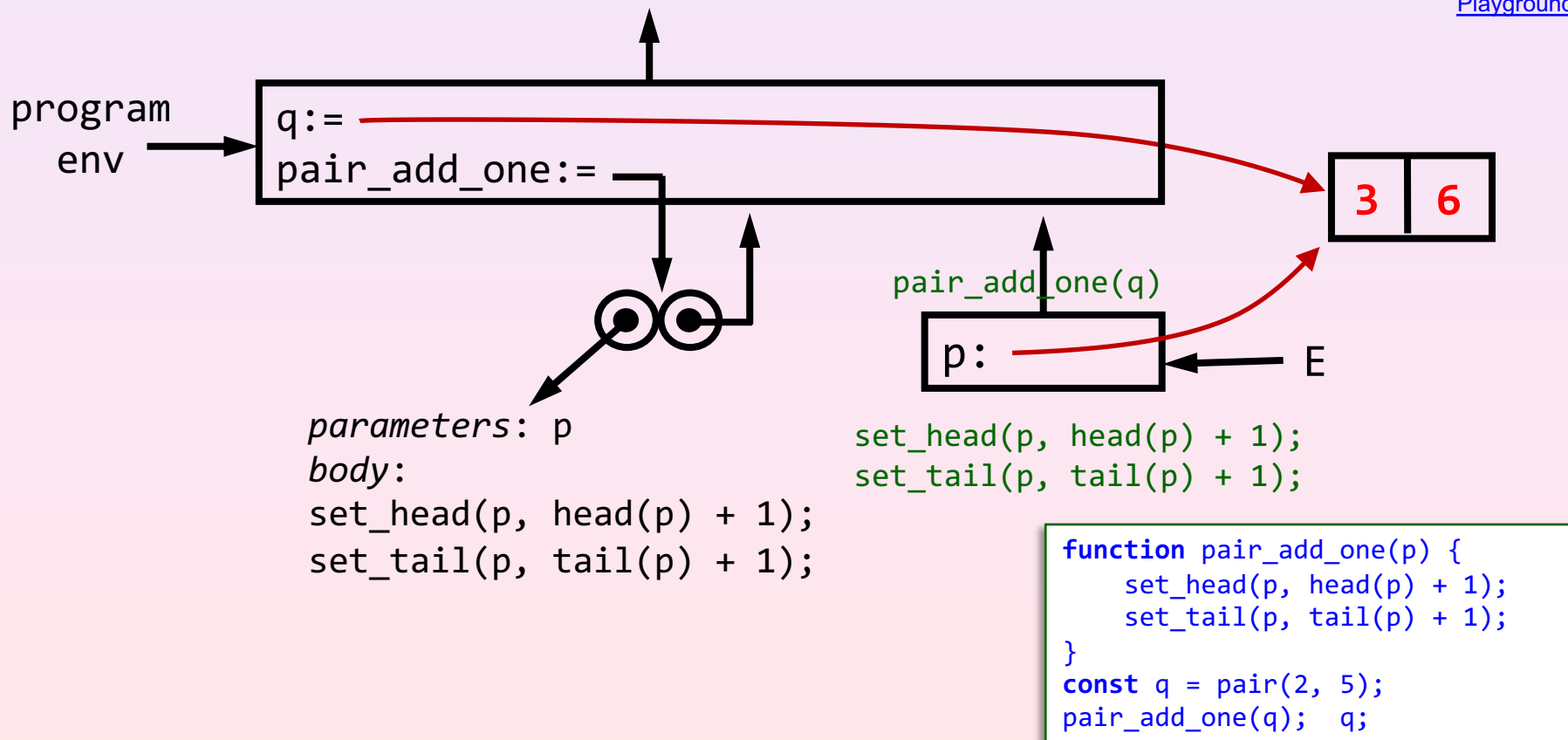
[Show in
Playground](#)



pair_add_one example

- Evaluate `set_head(p, head(p) + 1)` and `set_tail(p, tail(p) + 1)` in **Environment E**

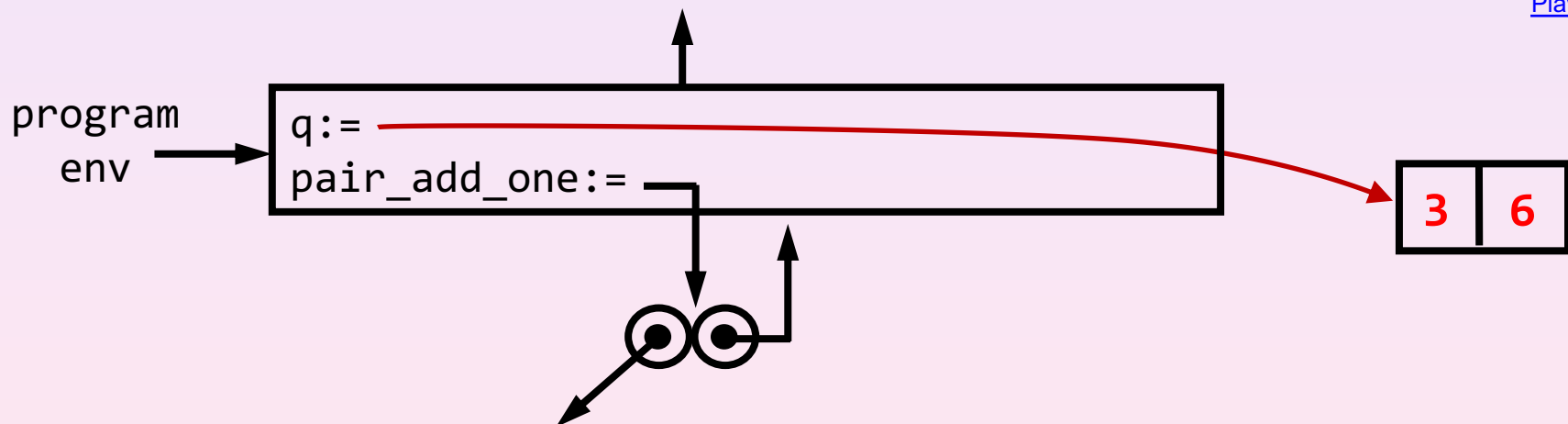
[Show in
Playground](#)



pair_add_one example

- Evaluate `q` in program environment

[Show in
Playground](#)



parameters: p
body:
set_head(p, head(p) + 1);
set_tail(p, tail(p) + 1);

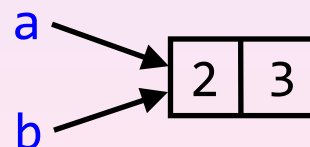
```
function pair_add_one(p) {  
    set_head(p, head(p) + 1);  
    set_tail(p, tail(p) + 1);  
}  
const q = pair(2, 5);  
pair_add_one(q); q;
```

Sharing and identity

- In `pair_add_one` example, `q` and `p` **share** the same **pair**
- Checking whether **two names** are **sharing** the same **pair**

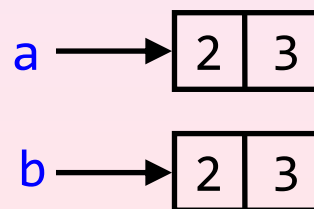
- **Example:**

```
const a = pair(2, 3);  
const b = a;  
a === b; ➔ true
```



- **Example:**

```
const a = pair(2, 3);  
const b = pair(2, 3);  
a === b; ➔ false
```



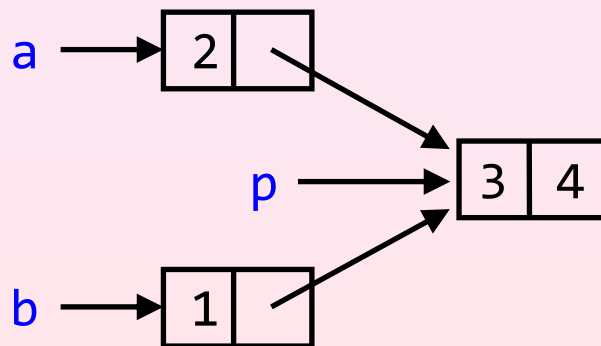
Sharing and identity

- **Structures** sharing pairs

- **Example:**

```
const p = pair(3, 4);  
const a = pair(2, p);  
const b = pair(1, p);  
tail(a) === tail(b); ➔ true
```

[Show in
Playground](#)

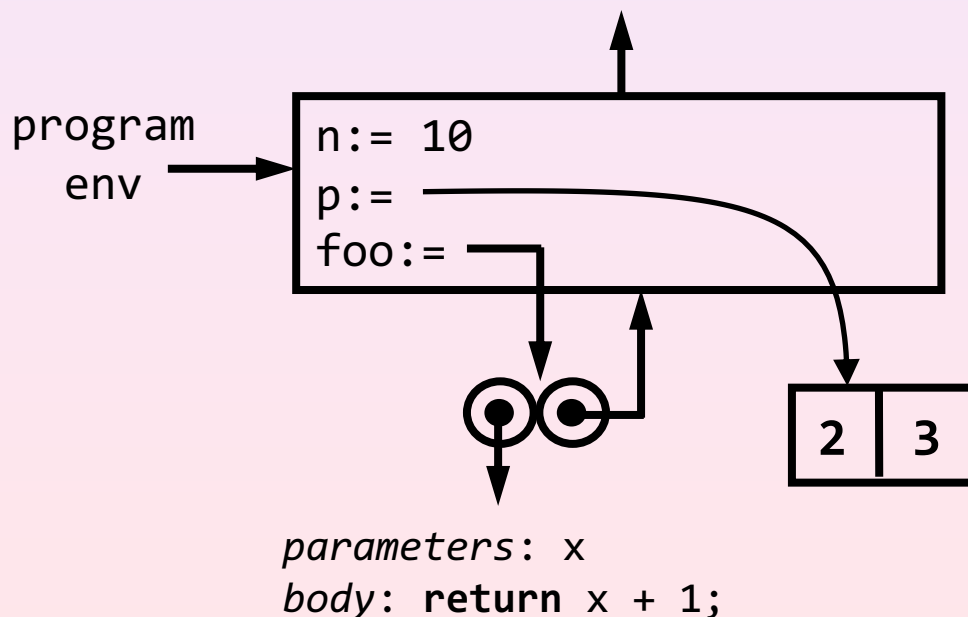


Identity in Source §3: ===

- For `a === b`, we say “is `a` *identical* to `b`”
 - `true`, `false`, `null`, `undefined` — each is identical to itself and nothing else
 - **Numbers** — two numbers are identical iff they have the same representation in the double-precision floating-point representation
 - **Strings** — two strings are identical if they have the same characters in the same order
 - **Functions** — functions are made by function expressions, and their creation bestows an identity upon them
 - **Pairs** — pairs are made by the `pair` function, and their creation bestows an identity upon them

Drawing compound structures

- **Primitive values** (e.g. numbers, strings, Boolean values, `null`) in bindings are drawn **inside** frames
- **Compound structures** (e.g. pairs, function objects) are drawn **outside** frames



Summary

- **Environments** keep track of name bindings
- A block creates a new **frame** for the names declared in the block
- When a **lambda expression** is evaluated, it creates a **function object** that remembers the **current environment**
- To evaluate a **function application**
 - 1) Create a **new frame** in the function's environment
 - 2) Bind **parameters** in new frame
 - 3) Evaluate **body** of function in the new environment
- **Compound values** live outside of environments and have **identity**